

Preprocessing

Ray Chandonnet and Barrett Viator

2025-08-13

```
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr    1.5.1
## v ggplot2     3.5.2      v tibble     3.3.0
## v lubridate  1.9.4      v tidyr      1.3.1
## v purrr       1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

library(ggplot2)
library(dplyr)
library(readxl)
library(writexl)
library(skimr)
library(knitr)
library(kableExtra)

##
## Attaching package: 'kableExtra'
##
## The following object is masked from 'package:dplyr':
##
##   group_rows

library(tidyr)
library(ggpmisc)

## Loading required package: ggpp
## Registered S3 methods overwritten by 'ggpp':
##   method                from
##   heightDetails.titleGrob ggplot2
##   widthDetails.titleGrob  ggplot2
##
## Attaching package: 'ggpp'
##
## The following object is masked from 'package:ggplot2':
##
##   annotate

library(forcats)
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(naniar)
```

```
##
```

```
## Attaching package: 'naniar'
```

```
##
```

```
## The following object is masked from 'package:skimr':
```

```
##
```

```
##      n_complete
```

```
library(reshape2)
```

```
##
```

```
## Attaching package: 'reshape2'
```

```
##
```

```
## The following object is masked from 'package:tidyr':
```

```
##
```

```
##      smiths
```

```
library(scales)
```

```
##
```

```
## Attaching package: 'scales'
```

```
##
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
##      discard
```

```
##
```

```
## The following object is masked from 'package:readr':
```

```
##
```

```
##      col_factor
```

```
library(stringr)
```

```
library(recipes)
```

```
##
```

```
## Attaching package: 'recipes'
```

```
##
```

```
## The following object is masked from 'package:stringr':
```

```
##
```

```
##      fixed
```

```
##
```

```
## The following object is masked from 'package:stats':
```

```
##
```

```
##      step
```

```
library(broom)
```

```
library(here)
```

```
## here() starts at /Users/raychandonnet/Chandonnet Family Dropbox/Chandonnet Family Folder/Ray Personal
```

```
library(rlang)
```

```
##
```

```
## Attaching package: 'rlang'
```

```
##
```

```

## The following objects are masked from 'package:purrr':
##
##   %%, flatten, flatten_chr, flatten_dbl, flatten_int, flatten_lgl,
##   flatten_raw, invoke, splice
library(rsample)
library(xts)

## Loading required package: zoo
##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##   as.Date, as.Date.numeric
##
## ##### Warning from 'xts' package #####
## #
## # The dplyr lag() function breaks how base R's lag() function is supposed to #
## # work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or #
## # source() into this session won't work correctly. #
## #
## # Use stats::lag() to make sure you're not using dplyr::lag(), or you can add #
## # conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop #
## # dplyr from breaking base R's lag() function. #
## #
## # Code in packages is not affected. It's protected by R's namespace mechanism #
## # Set `options(xts.warn_dplyr_breaks_lag = FALSE)` to suppress this warning. #
## #
## #####
##
## Attaching package: 'xts'
##
## The following objects are masked from 'package:dplyr':
##
##   first, last
library(randomForest)

## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##   combine
##
## The following object is masked from 'package:ggplot2':
##
##   margin
library(caTools)
library(caret)

```

```

## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:rsample':
##
##     calibration
##
## The following object is masked from 'package:purrr':
##
##     lift
library(RANN)
library(xgboost)

##
## Attaching package: 'xgboost'
##
## The following object is masked from 'package:dplyr':
##
##     slice
library(Matrix)

##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
library(e1071)

##
## Attaching package: 'e1071'
##
## The following object is masked from 'package:rsample':
##
##     permutations
PreProcessingData <- readRDS(here::here("Data","FirstCutData.rds")) # Get Data
stoplight <- c("#1a9641", "#ffea00", "#d7191c") # define heatmap colors
sleeptime <- 0.5
#####

# FUNCTIONS

wait_to_render <- function(seconds){
  Sys.sleep(seconds())
}

# Overall data missingness heatmap
OneCountryMissingness <- function(data, code) {

  data <- data %>%
    select(-Lag1_InternetUsersPct,
           -Lag2_InternetUsersPct,

```

```

      -Lag1_YearlyChgInternet,
      -Lag2_YearlyChgInternet,
      -Cumulative3yrChg_InternetUsersPct,
      -YearlyChgHDI,
      -YearlyChgInternet)

missing_heatmap_data <- data %>%
  filter(CountryCode == code) %>%
  select(-CountryCode, -CountryName, -IncomeGroup, -HDI_Category) %>%
  pivot_longer(cols = -year, names_to = "variable", values_to = "value") %>%
  mutate(Status = ifelse(is.na(value), "Missing", "Present"))

# Order variables by overall missingness (most missing at top)
var_order <- missing_heatmap_data %>%
  group_by(variable) %>%
  summarize(frac_missing = mean(Status == "Missing"), .groups = "drop") %>%
  arrange(dplyr::desc(frac_missing)) %>%
  pull(variable)

missing_heatmap_data <- missing_heatmap_data %>%
  mutate(variable = factor(variable, levels = var_order))

# 2) Plot: white = Present, red = Missing
print(
  ggplot(missing_heatmap_data, aes(x = year, y = variable, fill = Status)) +
    geom_tile(color = "white") +
    scale_fill_manual(values = c("Present" = "white", "Missing" = "red"), name = NULL) +
    labs(
      title = paste0("Data Availability Heatmap: ", code),
      x = "year",
      y = "variable"
    ) +
    theme_minimal(base_size = 11) +
    theme(
      panel.grid = element_blank(),
      axis.text.x = element_text(angle = 45, hjust = 1)
    )
)
wait_to_render(sleeptime)
}

OverallMissingnessHeatmap <- function(data) {
  data <- data %>%
    select(-Lag1_InternetUsersPct,
           -Lag2_InternetUsersPct,
           -Lag1_YearlyChgInternet,
           -Lag2_YearlyChgInternet,
           -Cumulative3yrChg_InternetUsersPct,
           -YearlyChgHDI,
           -YearlyChgInternet)
  # Step 1: Compute % missing per variable per year
  missing_heatmap_data <- data %>%
    group_by(year) %>%

```

```

summarize(across(everything(), ~mean(is.na(.)) * 100)) %>%
pivot_longer(-year, names_to = "variable", values_to = "pct_missing")

# Step 2: Plot with stoplight colors
print(ggplot(missing_heatmap_data, aes(x = year, y = variable, fill = pct_missing)) +
  geom_tile(color = "white") +
  scale_fill_gradientn(
    colors = stoplight,
    values = scales::rescale(c(0, 25, 100)),
    limits = c(0, 100),
    name = "% Missing"
  ) +
  labs(
    title = "Missing Data Heatmap",
    x = "Year",
    y = "Variable"
  ) +
  theme_minimal(base_size = 11) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    panel.grid = element_blank()
  ))
wait_to_render(sleeptime)
}

# Missingness chart by variable
MissingnessByVariable <- function(data) {
  PlotData <- data %>%
    select(-Lag1_InternetUsersPct,
           -Lag2_InternetUsersPct,
           -Lag1_YearlyChgInternet,
           -Lag2_YearlyChgInternet,
           -Cumulative3yrChg_InternetUsersPct,
           -YearlyChgHDI,
           -YearlyChgInternet)
  VariableMissing <- PlotData %>%
    summarize(across(everything(), ~ mean(is.na(.)) * 100)) %>%
    pivot_longer(cols = everything(), names_to = "Variable", values_to = "MissingPct")
  print(ggplot(data = VariableMissing,
    aes(x = reorder(Variable, MissingPct),
        y = MissingPct, fill = MissingPct)) +
    geom_col() +
    coord_flip() +
    scale_fill_gradientn(
      colors = stoplight,
      limits = c(0, 100),
      breaks = c(0, 25, 50, 75, 100), # include 0 and 100
      labels = c("0", "25", "50", "75", "100"),
      name = "% Missing"
    ) +
    labs(
      title = "Overall Sparseness by Variable",

```

```

    x = "Variable",
    y = "% Missing"
  ) +
  theme_minimal()
wait_to_render(sleeptime)
}

# Missingness chart by year
MissingnessByYear <- function(data) {
  PlotData <- data %>%
    select(-Lag1_InternetUsersPct,
           -Lag2_InternetUsersPct,
           -Lag1_YearlyChgInternet,
           -Lag2_YearlyChgInternet,
           -Cumulative3yrChg_InternetUsersPct,
           -YearlyChgHDI,
           -YearlyChgInternet)

  MissingYear <- PlotData %>%
    group_by(year) %>%
    summarize(across(everything(), ~ mean(is.na(.)) * 100)) %>%
    pivot_longer(-year, names_to = "variable", values_to = "pct_missing") %>%
    group_by(year) %>%
    summarize(pct_missing = mean(pct_missing)) %>%
    arrange(year)
  print(ggplot(data = MissingYear,
               aes(y = factor(year),
                   x = pct_missing, fill = pct_missing)) +
        geom_col() +
        scale_fill_gradientn(colors = stoplight,
                              values = rescale(c(0, 50, 100)),
                              name = "% Missing",
                              limits = c(0, 100),
                              breaks = c(0, 50, 100),
                              labels = c("0", "50", "100")) +
        labs(y = "Year",
              x = "% Missing (All Variables)",
              title = "Data Sparseness by Year") +
        theme_minimal() +
        theme(axis.text.y = element_text(size = 8),
              legend.position = "right")
  )
  wait_to_render(sleeptime)
}

MissingnessByCountry <- function(data, threshold) {
  PlotData <- data %>%
    select(-Lag1_InternetUsersPct,
           -Lag2_InternetUsersPct,
           -Lag1_YearlyChgInternet,
           -Lag2_YearlyChgInternet,
           -Cumulative3yrChg_InternetUsersPct,
           -YearlyChgHDI,

```

```

      -YearlyChgInternet)
CountryMissing <- PlotData %>%
  group_by(CountryCode) %>%
  summarize(across(everything(), ~ mean(is.na(.)) * 100)) %>%
  pivot_longer(-CountryCode, names_to = "variable", values_to = "pct_missing") %>%
  group_by(CountryCode) %>%
  summarize(pct_missing = mean(pct_missing)) %>%
  filter(pct_missing >= threshold) %>%
  arrange(pct_missing)
print(ggplot(data = CountryMissing,
             aes(y = reorder(CountryCode, pct_missing),
                 x = pct_missing, fill = pct_missing)) +
  geom_col() +
  scale_fill_gradientn(
    colors = stoplight,
    values = rescale(c(0, 50, 100)),
    name = "% Missing",
    limits = c(0, 100),
    breaks = c(0, 50, 100),
    labels = c("0", "50", "100")
  ) +
  labs(
    y = paste0("Country (Filtered > ", threshold, "%)"),
    x = "% Missing (All Variables)",
    title = "Data Sparseness by Country (High-Missingness Only)"
  ) +
  theme_minimal() +
  theme(
    axis.text.y = element_text(size = 6),
    legend.position = "right"
  ))
wait_to_render(sleeptime)
}

# This function just reproduces our four missingness plots
FourPlots <- function(data, cutoff) {
  MissingnessByVariable(data)
  MissingnessByYear(data)
  MissingnessByCountry(data, cutoff)
  OverallMissingnessHeatmap(data)
}

# This function plots average UHCServiceCoverage by year
PlotUHC <- function(data, sleeptime = 0) {
  data_to_plot <- data %>%
    mutate(year = as.numeric(year)) %>%
    group_by(year) %>%
    summarize(
      avg_uhc = mean(UHCServiceCoverage, na.rm = TRUE),
      n_countries = sum(!is.na(UHCServiceCoverage)),
      .groups = "drop"
    ) %>%
    filter(!is.na(avg_uhc)) %>%

```



```

    arrange(year)

plot <- ggplot(data_to_plot, aes(x = year, y = avg_uhc)) +
  geom_line(color = "steelblue", linewidth = 1) +
  geom_point(aes(size = n_countries), color = "steelblue") +
  scale_x_continuous(breaks = seq(min(data_to_plot$year),
                                   max(data_to_plot$year), by = 1)) +

  labs(
    title = "Average UHC Service Coverage by Year",
    x = "Year",
    y = "Average UHC Coverage Index",
    size = "Countries with data"
  ) +
  theme_minimal()

print(plot)
Sys.sleep(sleeptime)
}

# Function to Plot PopInSlums by year
PlotSlums <- function(data){
  data_to_plot <- data %>%
    mutate(year = as.numeric(year)) %>%
    group_by(year) %>%
    summarize(
      avg_slums = mean(PopInSlums, na.rm = TRUE),
      n_countries = sum(!is.na(PopInSlums))
    ) %>%
    filter(!is.na(avg_slums)) %>%
    arrange(year)

plot <- ggplot(data_to_plot, aes(x = year, y = avg_slums)) +
  geom_line(color = "steelblue", linewidth = 1) +
  geom_point(aes(size = n_countries), color = "steelblue") +
  scale_x_continuous(breaks = seq(min(data_to_plot$year),
                                   max(data_to_plot$year), by = 1)) +

  labs(title = "Average Population in Slums by Year", x = "Year",
       y = "Average Population in Slums", size = "Countries with data") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
print(plot)
wait_to_render(sleeptime)
}

# Yeo-Johnson Transformation to handle skewness. This function creates the
# recipe, preps it and returns the prepped recipe. That prepped recipe will
# be used to "bake" both the training and

Yeo_Johnson <- function(train_data, test_data) {

  # Get affected columns from training data
  affected_cols <- train_data %>%
    select(InternetUsersPct,

```

```

        Lag1_InternetUsersPct,
        Lag2_InternetUsersPct,
        YearlyChgInternet,
        Lag1_YearlyChgInternet,
        Lag2_YearlyChgInternet) %>%
names()

# Preprocessing recipe - fit only on training data
data_recipe <- recipe(~ ., data = train_data) %>%
  update_role(CountryName, CountryCode, new_role = "id") %>%
  step_YeoJohnson(all_of(affected_cols), na_rm = TRUE)

# Prep the recipe on training data
transformed_recipe <- prep(data_recipe, training = train_data)

# Bake the data
train_transformed <- bake(transformed_recipe, new_data = train_data)
test_transformed <- bake(transformed_recipe, new_data = test_data)

# Yeo-Johnson Output
cat("\nEstimated lambdas for transformed variables:\n")
print(transformed_recipe$steps[[1]]$lambdas)

# Histogram comparisons for training data
par(mfrow = c(1, 2))
hist(train_data$InternetUsersPct,
     main = "Train: Original Internet Users",
     xlab = "Original Values")
hist(train_transformed$InternetUsersPct,
     main = "Train: Transformed Internet Users",
     xlab = "Transformed Values")

# Histogram comparisons for test data
par(mfrow = c(1, 2))
hist(test_data$InternetUsersPct,
     main = "Test: Original Internet Users",
     xlab = "Original Values")
hist(test_transformed$InternetUsersPct,
     main = "Test: Transformed Internet Users",
     xlab = "Transformed Values")

par(mfrow = c(1, 1)) # reset layout

# Output first 5 rows of transformed training data
cat("\nFirst 5 rows of Transformed Training Data:\n")
print(head(train_transformed))

# Return both datasets
return(list(
  train = train_transformed,
  test = test_transformed
))
}

```

```

#
# This function imputes missing data using time trend fitting, as follows:
# The variable name is passed as an argument so the function can work with
# multiple variables. A floor and ceiling value are also passed
#
# For the missing data for each CountryCode group:
#
# If there are 3 or more valid data points, impute the rest quadratically
# If there are 2 valid data points, impute the rest quadratically
# If there is 1 valid data point, carry it throughout the rest of the observations
# If there are none, leave it as NA
# Read in the data produced in the EDA exercise

impute_time_trend <- function(data, var_name, floor, cap) {
  var_sym <- rlang::ensym(var_name)

  data %>%
    group_by(CountryName) %>%
    group_modify(~ {
      df_country <- .x
      y <- df_country[[as_name(var_sym)]]
      known <- df_country %>% filter(!is.na(y))

      if (nrow(known) >= 3) {
        model <- lm(y ~ poly(year, 2, raw = TRUE), data = df_country)
        preds <- predict(model, newdata = df_country)
      } else if (nrow(known) == 2) {
        model <- lm(y ~ year, data = df_country)
        preds <- predict(model, newdata = df_country)
      } else if (nrow(known) == 1) {
        preds <- rep(known[[rlang::as_name(var_sym)]][1], nrow(df_country))
      } else {
        preds <- rep(NA_real_, nrow(df_country))
      }

      missing_idx <- is.na(y) & !is.na(preds)
      preds_clipped <- pmin(cap, pmax(floor, preds[missing_idx]))
      df_country[[rlang::as_name(var_sym)]][missing_idx] <- preds_clipped
      df_country
    }) %>%
    ungroup()
}

### FIRST - DO ALL THE PREPROCESSING THAT CAN BE DONE ON THE ENTIRE SET
### (BEFORE SPLITTING INTO TESTING AND TRAINING)
#
# Important to note that since we are splitting our series by country,
# retain all years' data for an individual country together in either
# training or test (no country will have data in both sets, we can apply
# all of our imputation methods to the entire data set because none of those
# methods imputes data across countries - it's all WITHIN a country's time series
# Only the Yeo-Johnson transforms and PCA are done separately since we need to apply
# the results of these obtained from the training data onto the test data to avoid

```

```

# leakage and ensure we are using the same modeling approach

#####
#
# Drop GiniCoeff, IHDI_Index and FixedIntSUBS. The first 2 we aren't
# using in the first go-round, and the the last one we may not get to use at all (it
# shows fixed internet subscriptions, which is only part of the picture of internet use.
# In the first models we will just use InternetUsersPct or its derivatives

PreProcessingData <- PreProcessingData |>
  select(-IHDI_Index, -GiniCoeff, -FixedIntSubs)

# Rename the Income Group column to remove the space - just good naming convention
PreProcessingData <- PreProcessingData |>
  rename(IncomeGroup = `Income Group`)

# Create additional variables for change in Internet access and HDI,
# as they might correlate better and/or indicative causality

PreProcessingData <- PreProcessingData |>
  arrange(CountryCode, year) |>
  group_by(CountryCode) |>
  mutate(
    # Internet access. (alternative predictor)
    YearlyChgInternet = InternetUsersPct - lag(InternetUsersPct),
    # HDI (alternative response)
    YearlyChgHDI = HDI_Index - lag(HDI_Index),
  ) |>
  ungroup()

# Create additional features for 1 and 2 year time lag, and for 3 year cumulative change in
# internet access, the idea being that it takes time for changes in internet access that to
# flow through to HDI

PreProcessingData <- PreProcessingData |>
  arrange(CountryCode, year) |>
  group_by(CountryCode) |>
  mutate(
    Lag1_InternetUsersPct = lag(InternetUsersPct, 1),
    Lag2_InternetUsersPct = lag(InternetUsersPct, 2),

    Lag1_YearlyChgInternet = lag(YearlyChgInternet, 1),
    Lag2_YearlyChgInternet = lag(YearlyChgInternet, 2),

    Cumulative3yrChg_InternetUsersPct = InternetUsersPct - lag(InternetUsersPct, 3)
  ) |>
  ungroup()

# Set up HDI Bins
PreProcessingData <- PreProcessingData %>%
  mutate(
    HDI_Category = case_when(
      HDI_Index >= 0.800 ~ "Very high human development",

```

```

    HDI_Index >= 0.700 & HDI_Index <= 0.799 ~ "High human development",
    HDI_Index >= 0.550 & HDI_Index <= 0.699 ~ "Medium human development",
    TRUE ~ "Low human development"
  )
)

# move the position of the new classified HDI data to a more relevant spot
PreProcessingData <- PreProcessingData %>%
  relocate(HDI_Category, .after = HDI_Index)

print("\nData after HDI classification:")

## [1] "\nData after HDI classification:"

print(PreProcessingData)

## # A tibble: 5,974 x 40
##   CountryCode CountryName year HDI_Index HDI_Category FoodIndex
##   <chr>        <chr>    <int>    <dbl> <chr>          <dbl>
## 1 AFG          Afghanistan 1995    0.329 Low human development 67.9
## 2 AFG          Afghanistan 1996    0.334 Low human development 71.4
## 3 AFG          Afghanistan 1997    0.338 Low human development 76.4
## 4 AFG          Afghanistan 1998    0.338 Low human development 80.6
## 5 AFG          Afghanistan 1999    0.347 Low human development 79.7
## 6 AFG          Afghanistan 2000    0.351 Low human development 67.8
## 7 AFG          Afghanistan 2001    0.355 Low human development 61.6
## 8 AFG          Afghanistan 2002    0.383 Low human development 74.7
## 9 AFG          Afghanistan 2003    0.392 Low human development 76.8
## 10 AFG         Afghanistan 2004    0.408 Low human development 74.8
## # i 5,964 more rows
## # i 34 more variables: CorruptionScore <dbl>, ElectricAccess <dbl>,
## #   PopDensity <dbl>, PopInSlums <dbl>, WaterStress <dbl>, BankingAccess <dbl>,
## #   RndInvest <dbl>, GovtEffectiveness <dbl>, ShippingIndex <dbl>,
## #   InternetUsersPct <dbl>, GDPPerCapGrowth <dbl>, GDPGrowth <dbl>, GDP <dbl>,
## #   PoliticalStability <dbl>, RuleOfLaw <dbl>, RqdEduYears <dbl>,
## #   GovtEduSpendPctGDP <dbl>, MortalityFromDirtness <dbl>, ...

# Combined Governance Score to address collinearity
PreProcessingData <- PreProcessingData %>%
  mutate(GovernanceScore = rowMeans(pick(GovtEffectiveness,
                                          RuleOfLaw, CorruptionScore),
                                          na.rm = TRUE)) %>%
  mutate(HealthSpend = rowMeans(pick(HealthSpendPerCapita,
                                     GovtHealthSpendPerCapita),
                                     na.rm = TRUE))

PreProcessingData <- PreProcessingData %>%
  select(-GovtEffectiveness,
        -RuleOfLaw,
        -CorruptionScore,
        -HealthSpendPerCapita,
        -GovtHealthSpendPerCapita)

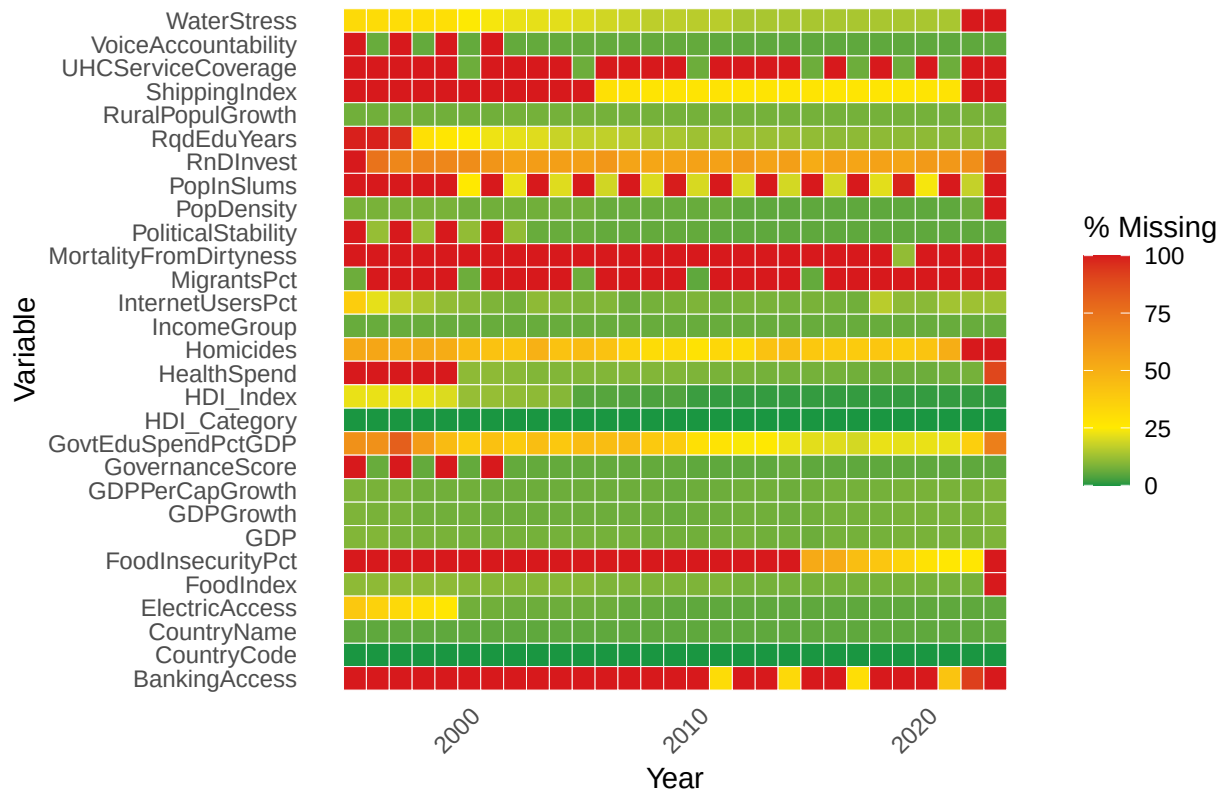
print("\nData with the new columns:")

```

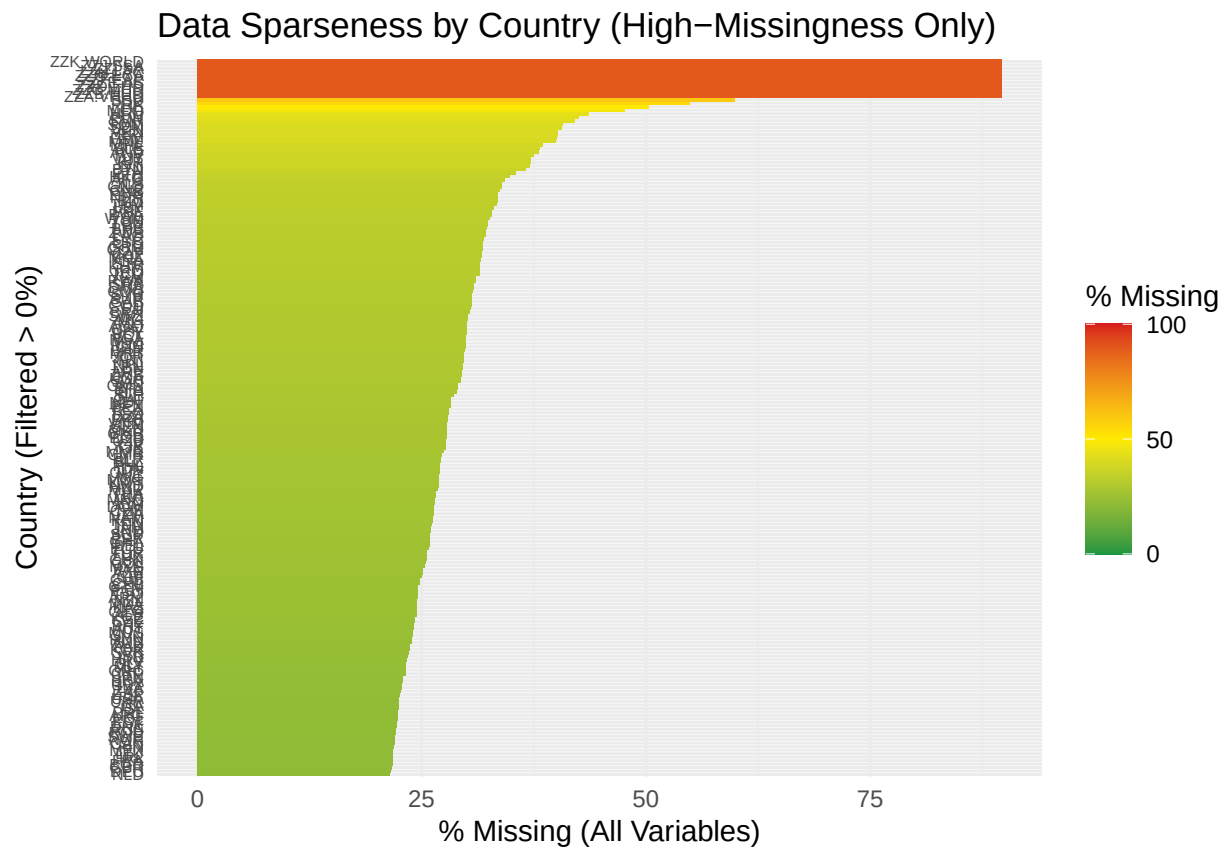
```
## [1] "\nData with the new columns:"
print(PreProcessingData)

## # A tibble: 5,974 x 37
##   CountryCode CountryName  year HDI_Index HDI_Category FoodIndex ElectricAccess
##   <chr>         <chr>      <int>    <dbl> <chr>          <dbl>          <dbl>
## 1 AFG          Afghanistan 1995    0.329 Low human d~    67.9          NaN
## 2 AFG          Afghanistan 1996    0.334 Low human d~    71.4          NaN
## 3 AFG          Afghanistan 1997    0.338 Low human d~    76.4          NaN
## 4 AFG          Afghanistan 1998    0.338 Low human d~    80.6          NaN
## 5 AFG          Afghanistan 1999    0.347 Low human d~    79.7          NaN
## 6 AFG          Afghanistan 2000    0.351 Low human d~    67.8          4.4
## 7 AFG          Afghanistan 2001    0.355 Low human d~    61.6          9.3
## 8 AFG          Afghanistan 2002    0.383 Low human d~    74.7         14.1
## 9 AFG          Afghanistan 2003    0.392 Low human d~    76.8          19
## 10 AFG         Afghanistan 2004    0.408 Low human d~    74.8         23.8
## # i 5,964 more rows
## # i 30 more variables: PopDensity <dbl>, PopInSlums <dbl>, WaterStress <dbl>,
## #   BankingAccess <dbl>, RnDInvest <dbl>, ShippingIndex <dbl>,
## #   InternetUsersPct <dbl>, GDPPerCapGrowth <dbl>, GDPGrowth <dbl>, GDP <dbl>,
## #   PoliticalStability <dbl>, RqdEduYears <dbl>, GovtEduSpendPctGDP <dbl>,
## #   MortalityFromDirtness <dbl>, UHCServiceCoverage <dbl>, MigrantsPct <dbl>,
## #   FoodInsecurityPct <dbl>, RuralPopulGrowth <dbl>, ...
#####
#
# DEALING WITH MISSING DATA - OVERALL
#
# Show missingness heatmap
OverallMissingnessHeatmap(PreProcessingData)
```

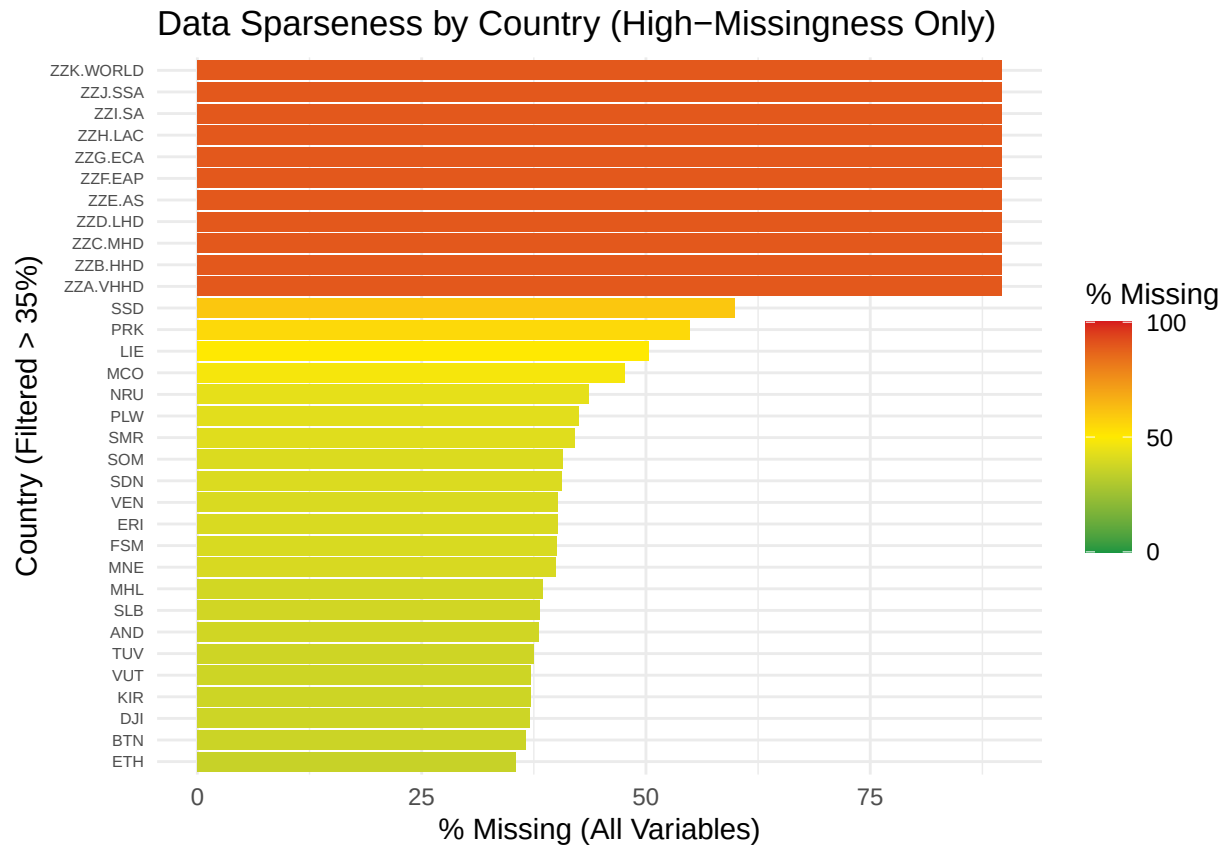
Missing Data Heatmap



```
# Plot missingness for all countries
MissingnessByCountry(PreProcessingData, 0)
```



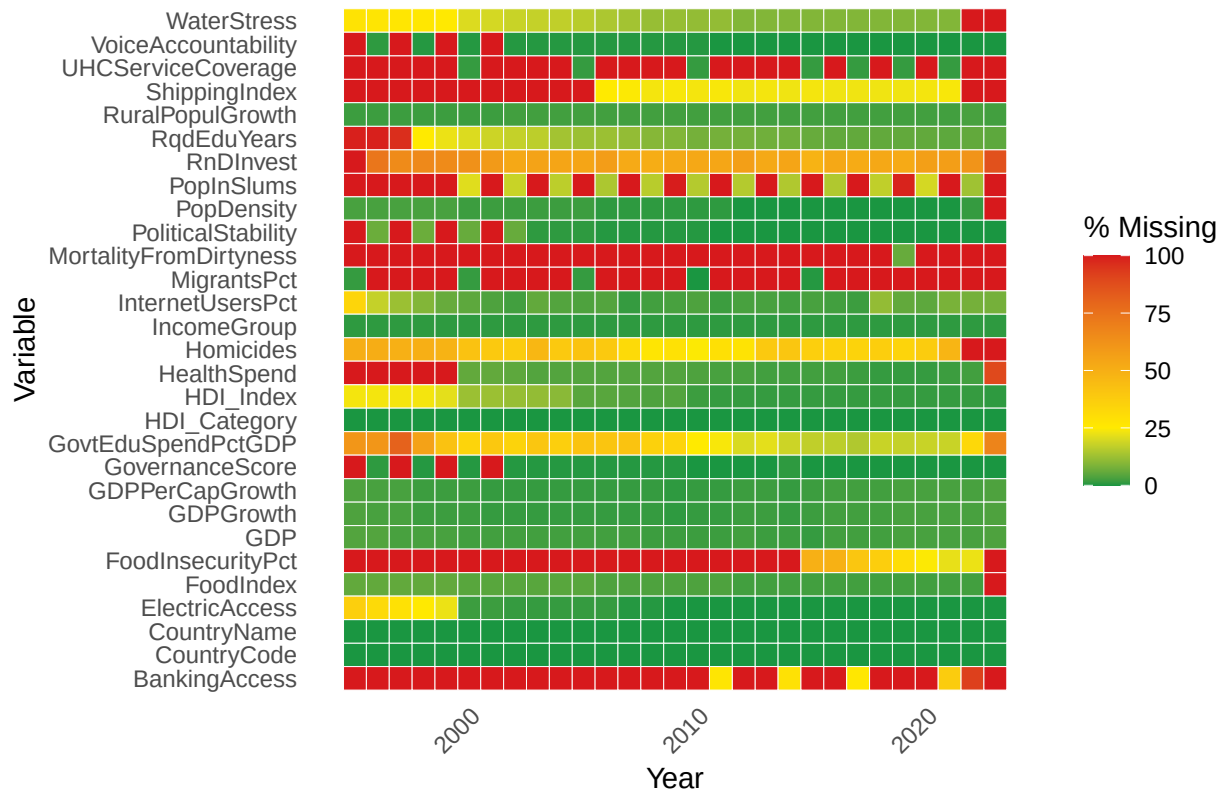
```
# Zero in on subset > 35%  
MissingnessByCountry(PreProcessingData, 35)
```

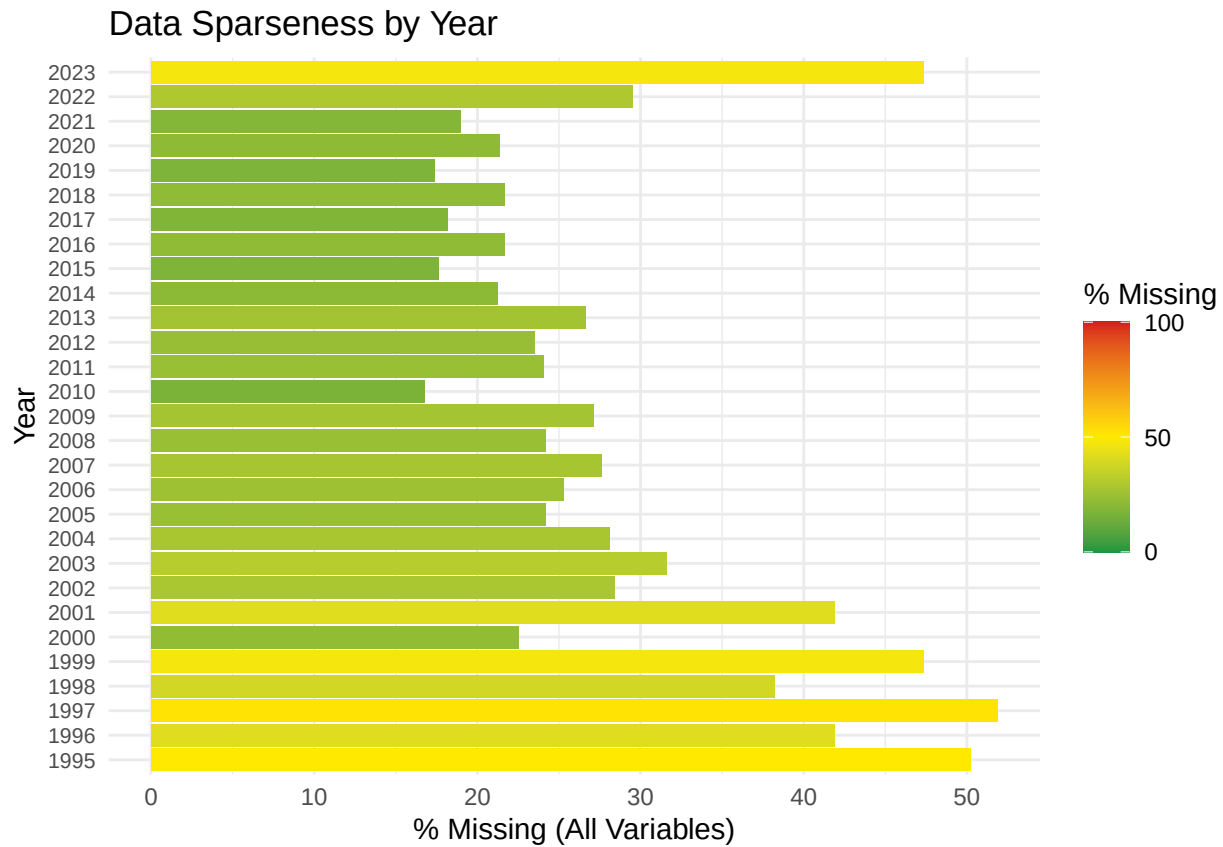
```
# Delete all the rows with country code starting with ZZ - virtually no data
PreProcessingData <- PreProcessingData %>%
  filter(!str_starts(CountryCode, "ZZ"))

# Show missingness heatmap
OverallMissingnessHeatmap(PreProcessingData)
```

Missing Data Heatmap

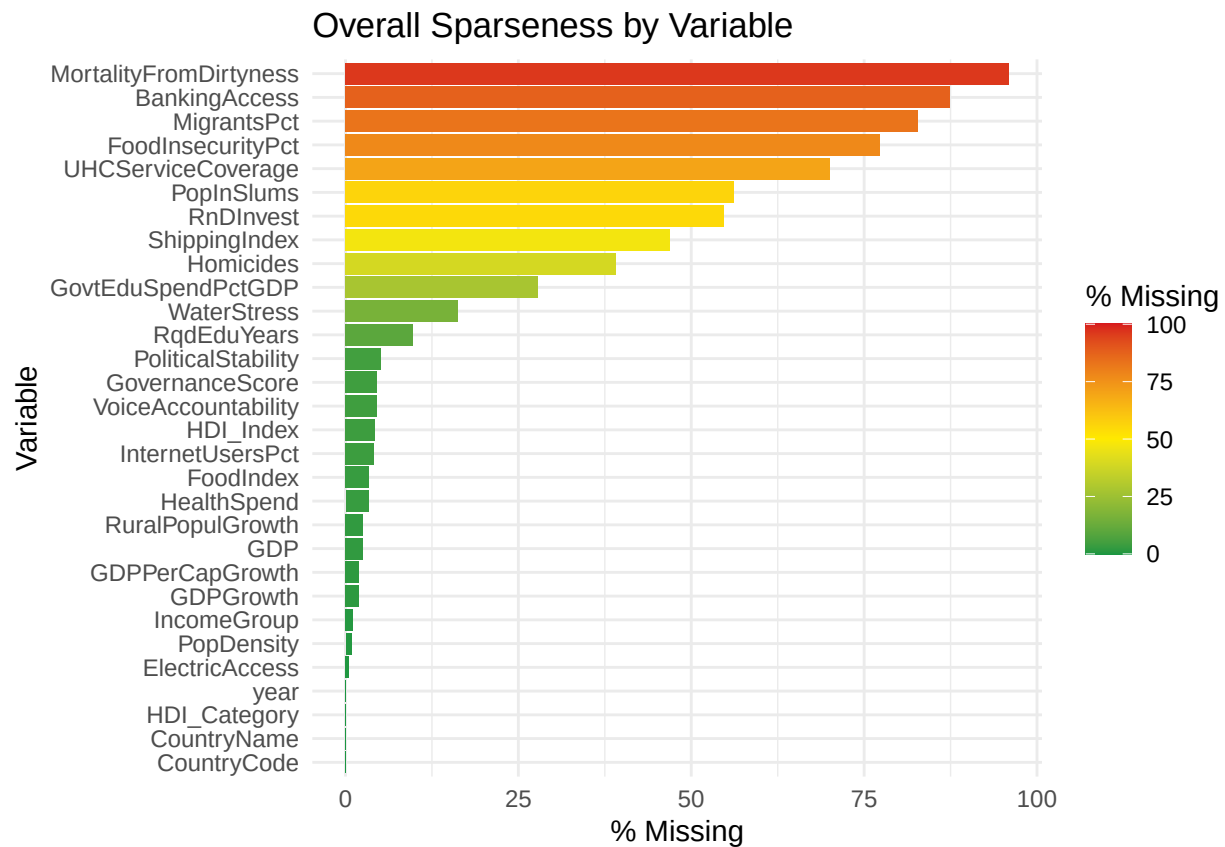


```
# Re-examine missingness by year
MissingnessByYear(PreProcessingData)
```



```
# Delete 1995-1999 and 2023 (too sparse)
PreProcessingData <- PreProcessingData %>%
  filter(!(year %in% c(1995, 1996, 1997, 1998, 1999, 2023)))

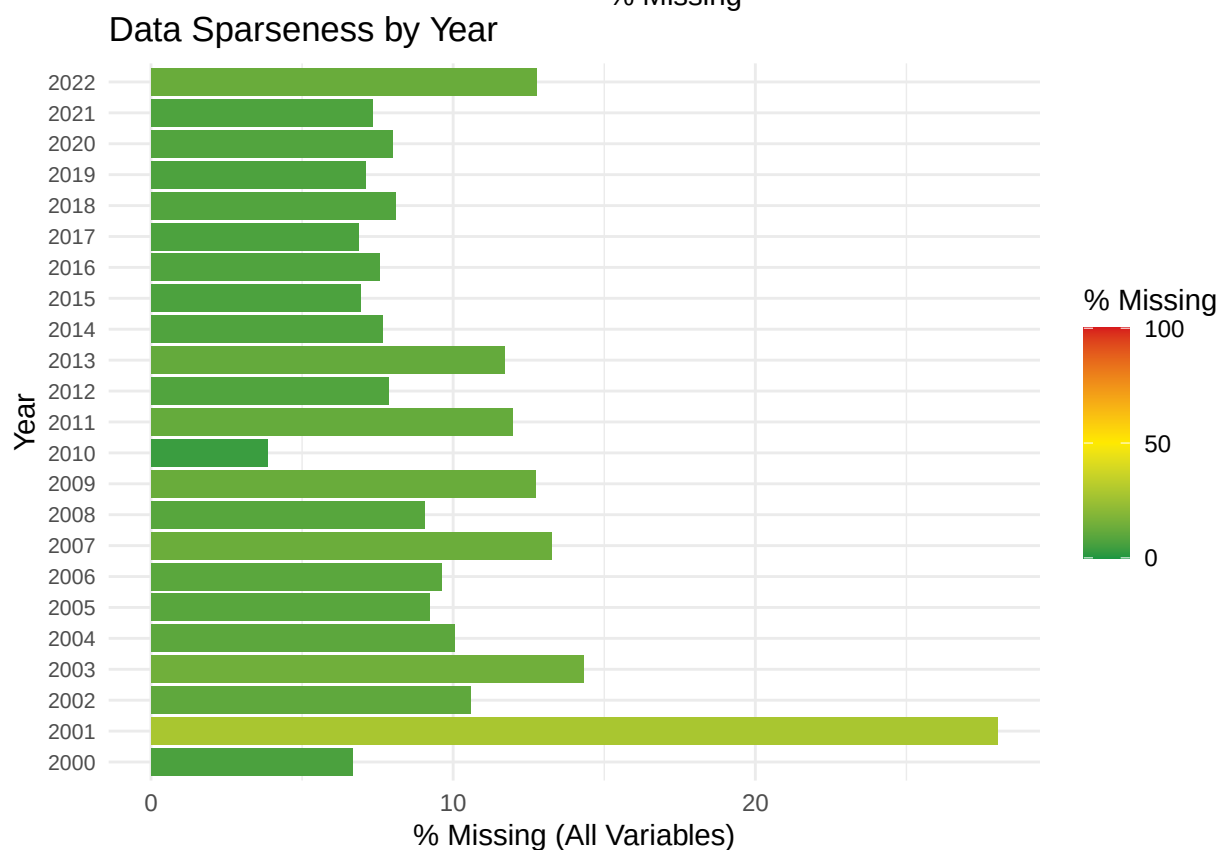
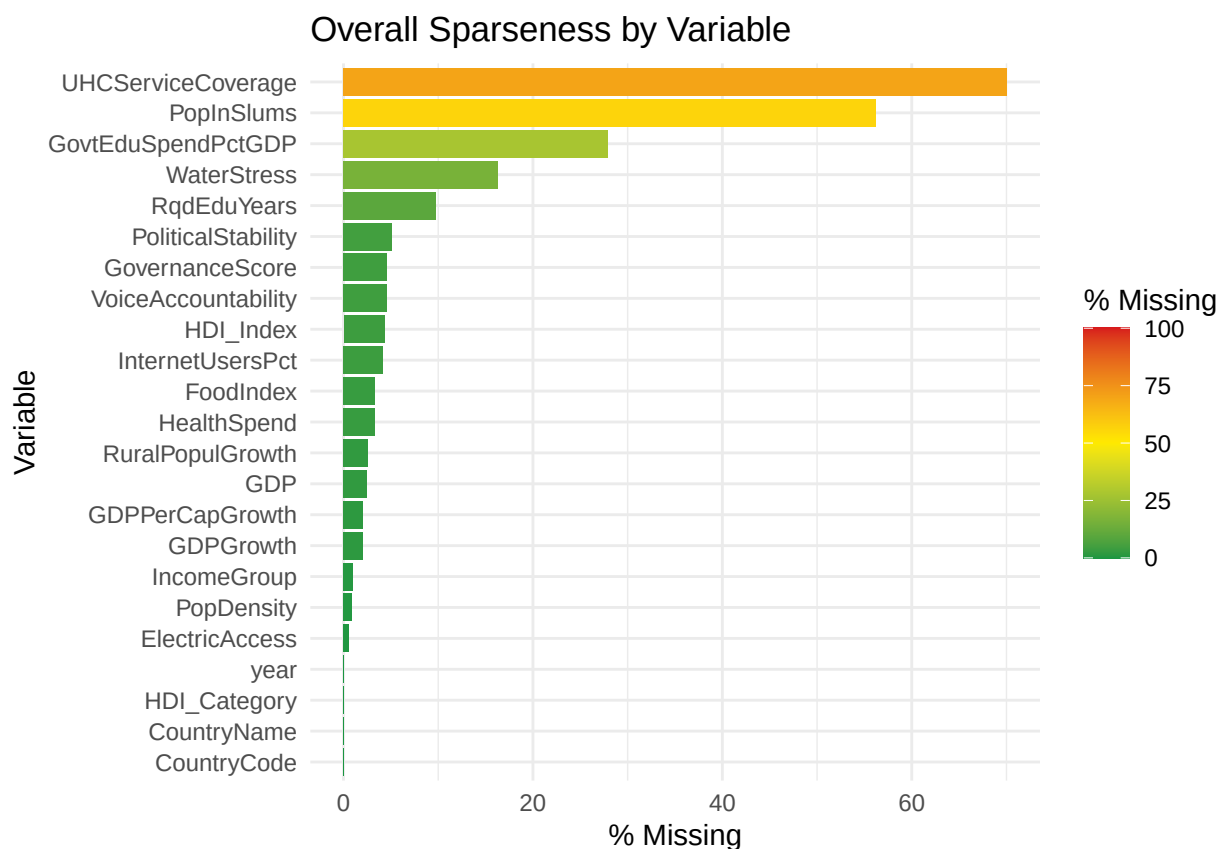
# Review missingness by variable
MissingnessByVariable(PreProcessingData)
```



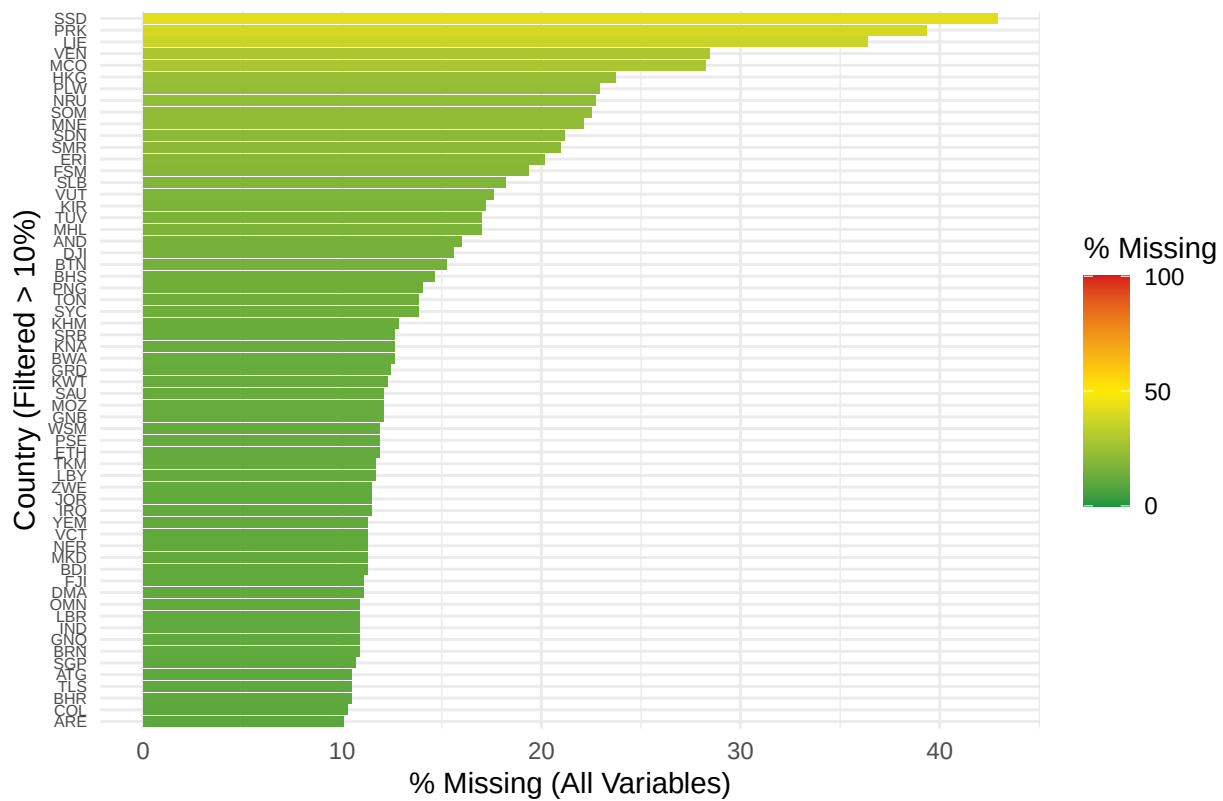
```
# Delete the features that have over 25% missing data with no
# pattern to the missingness
```

```
PreProcessingData <- PreProcessingData %>%
  select(-MortalityFromDirtness,
    -BankingAccess,
    -MigrantsPct,
    -FoodInsecurityPct,
    -RnDInvest,
    -ShippingIndex,
    -Homicides)
```

```
# Rerun all charts. Now the real work begins
FourPlots(PreProcessingData,10)
```



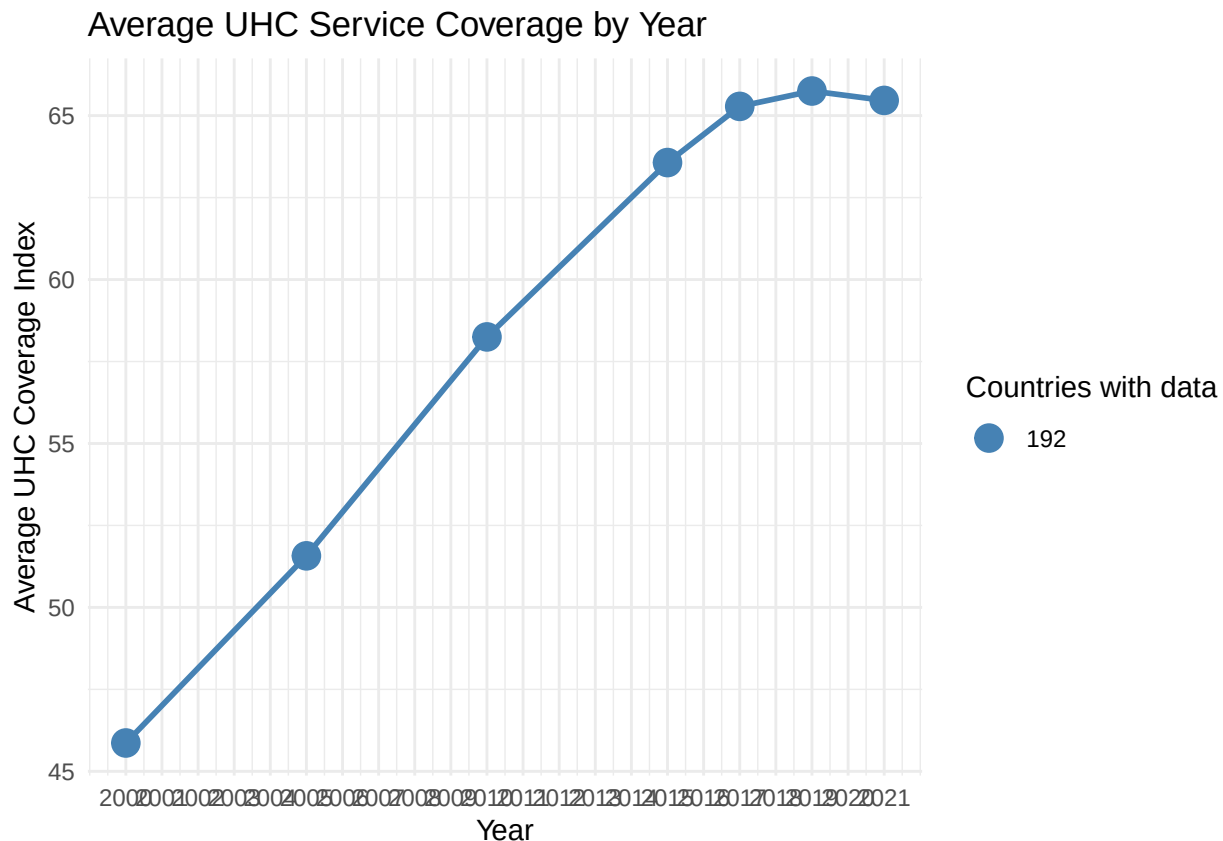
Data Sparseness by Country (High-Missingness Only)



Missing Data Heatmap



```
# First we'll tackle UHCServiceCoverage which has a pattern to its missingness.
# We can interpolate the missing late years 92015 on) - can we interpolate the
# big blocks of missing data when it was only being reported every 5 years?
# We Plot the UHC curve to see if it's linear enough to do so
PlotUHC(PreProcessingData)
```



```
# Since the plot between 2000 and 2015 is pretty straight, we ARE going to linearly
# interpolate all the missing values where we can. If this variable turns out to be
# critically important to our predictive model we'll have to note that!
```

```
anchor_years <- c(2000, 2005, 2010, 2015)
PreProcessingData <- PreProcessingData %>%
  arrange(CountryCode, year) %>%
  group_by(CountryCode) %>%
  group_modify(~{
    df <- .x
    anchor_rows <- df$year %in% anchor_years & !is.na(df$UHCServiceCoverage)

    if (sum(anchor_rows) >= 2) {
      interp <- approx(
        x = df$year[anchor_rows],
        y = df$UHCServiceCoverage[anchor_rows],
        xout = df$year,
        method = "linear",
        rule = 1
      )$y
    }
  })
```

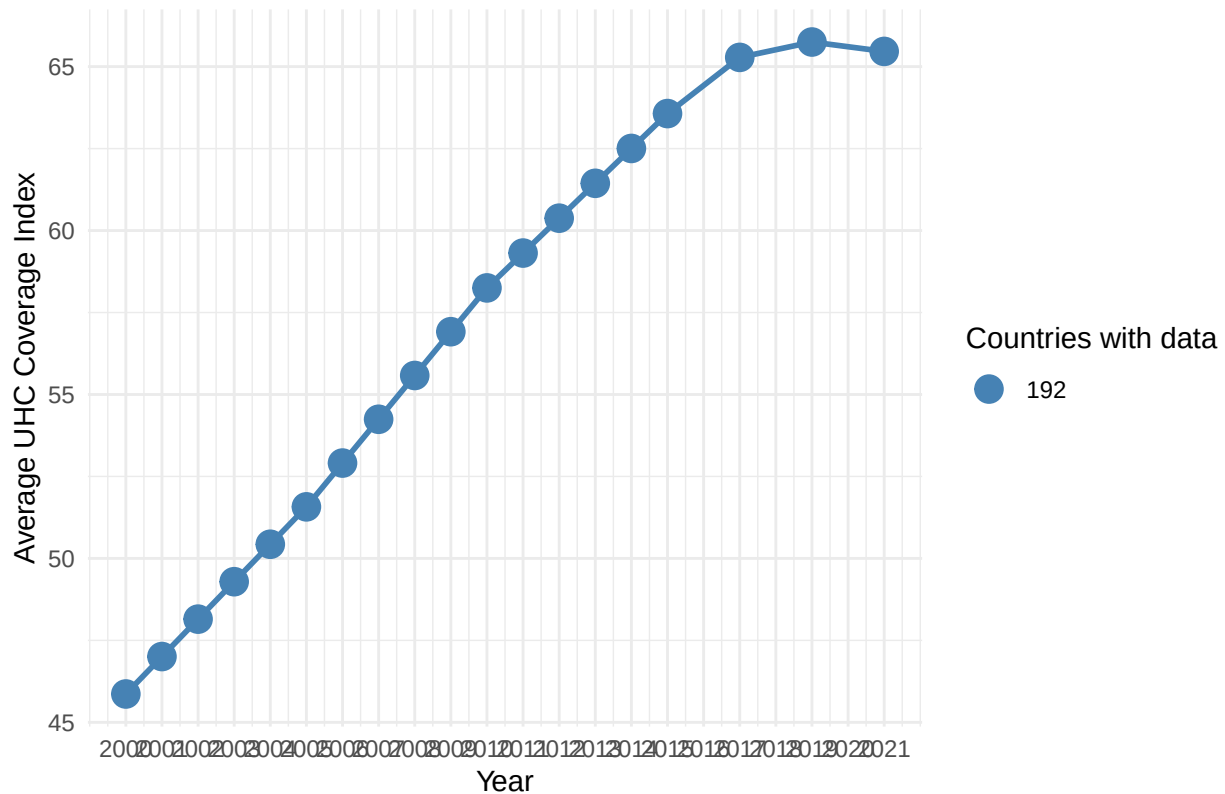
```

# replace missing values only
df$UHCServiceCoverage[is.na(df$UHCServiceCoverage)] <- interp[is.na(df$UHCServiceCoverage)]
}
df
}) %>%
ungroup()

# plot the curve again to make sure it did it right
PlotUHC(PreProcessingData)

```

Average UHC Service Coverage by Year



```

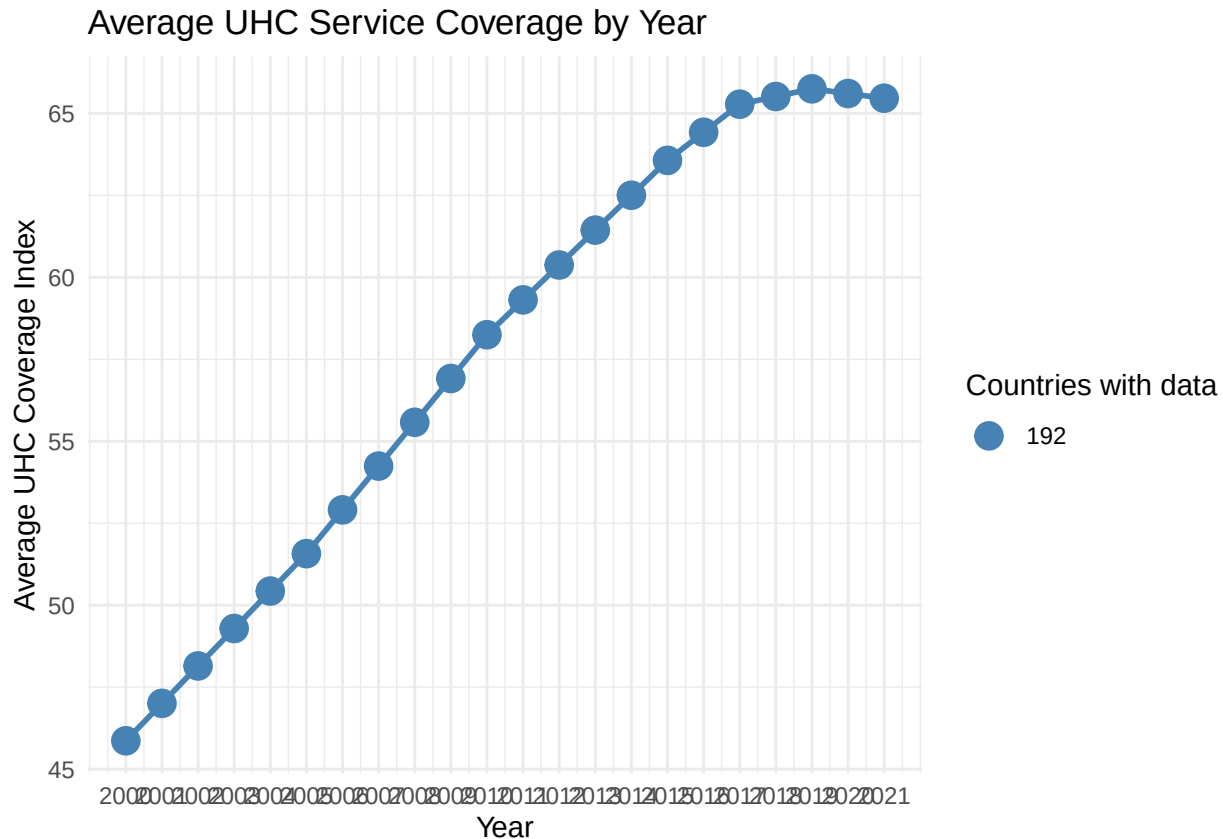
# Now - interpolate the years 2016, 2018 and 2020
PreProcessingData <- PreProcessingData %>%

group_by(CountryCode) %>%
mutate(
  prev_val = UHCServiceCoverage[match(year - 1, year)],
  next_val = UHCServiceCoverage[match(year + 1, year)],
  UHCServiceCoverage = if_else(
    year %in% c(2016, 2018, 2020) &
    is.na(UHCServiceCoverage) &
    !is.na(prev_val) & !is.na(next_val),
    (prev_val + next_val) / 2,
    UHCServiceCoverage
  )
) %>%
select(-prev_val, -next_val) %>%
ungroup()

```



```
# plot the curve again to make sure it did it right
PlotUHC(PreProcessingData)
```



```
# Now extrapolate missing 2022 using 2019-2021
PreProcessingData <- PreProcessingData %>%
  group_by(CountryCode) %>%
  group_modify(~ {
    this_group <- .x

    # Try to fit model if at least 2 of 2019-2021 are present
    recent_years <- c(2019, 2020, 2021)
    recent_data <- this_group %>%
      filter(year %in% recent_years & !is.na(UHCServiceCoverage))

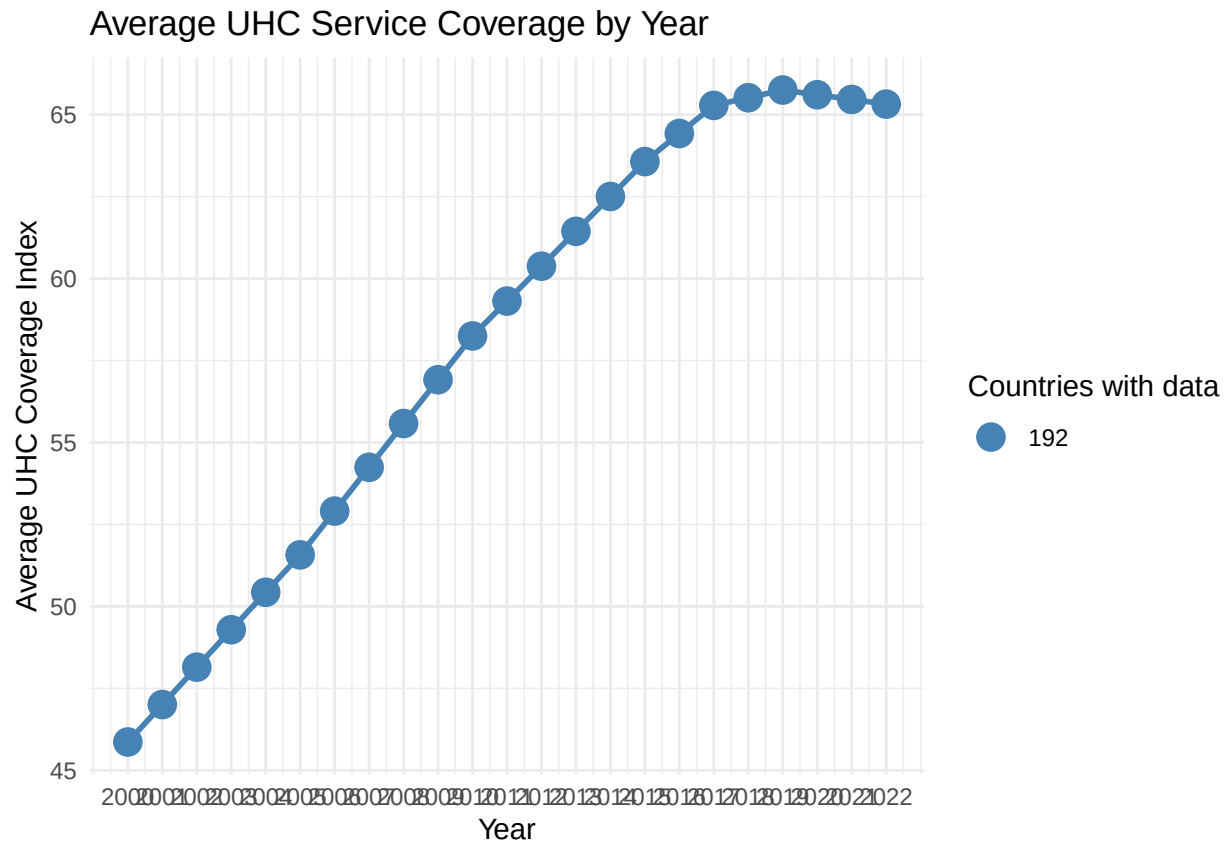
    # If enough data to fit a line
    if (nrow(recent_data) >= 2) {
      model <- lm(UHCServiceCoverage ~ year, data = recent_data)
      predicted_2022 <- predict(model, newdata = data.frame(year = 2022))[1]

      # Fill in 2022 if it's NA
      this_group <- this_group %>%
        mutate(UHCServiceCoverage = ifelse(year == 2022 & is.na(UHCServiceCoverage),
                                             predicted_2022,
                                             UHCServiceCoverage))
    }

    this_group
  })
```

```
} %>%
ungroup()
```

```
# plot the curve again to make sure it did it right
PlotUHC(PreProcessingData)
```



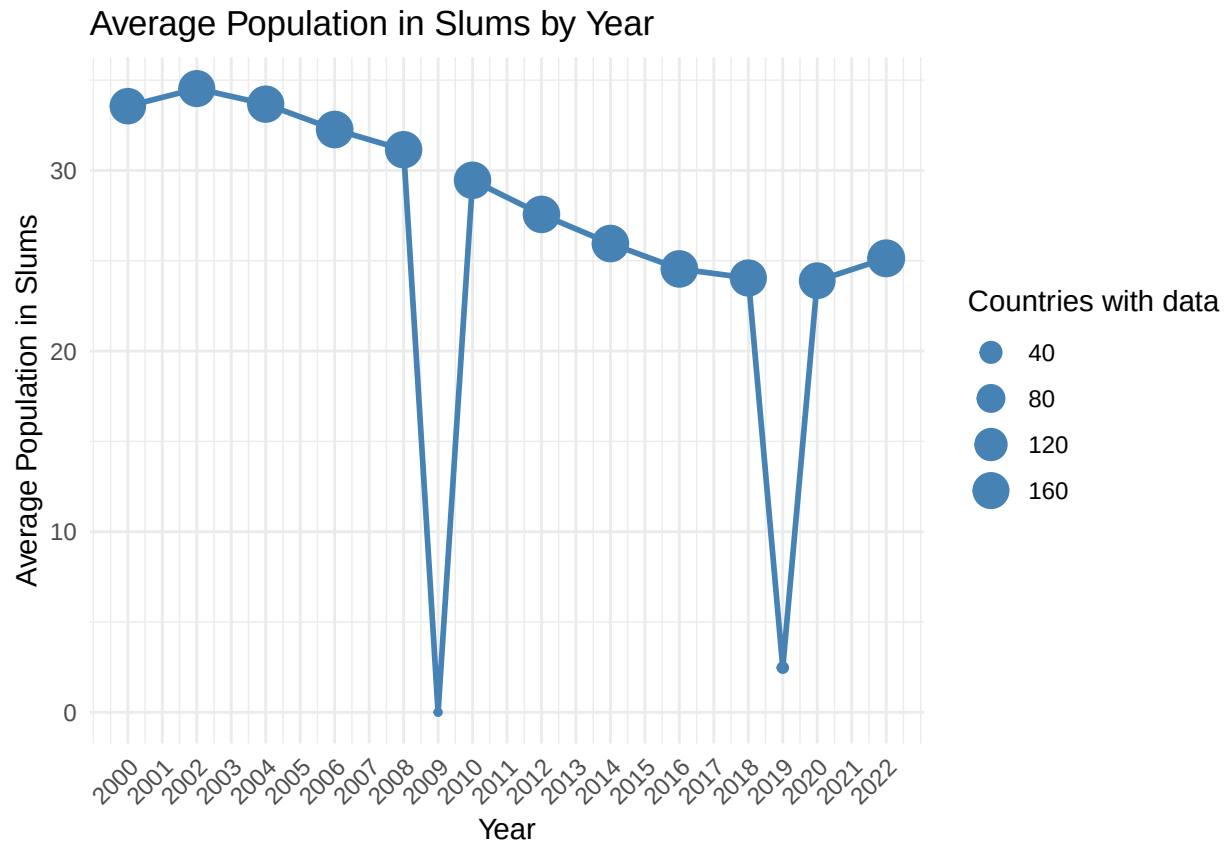
```
# Show the heatmap again
OverallMissingnessHeatmap(PreProcessingData)
```

Missing Data Heatmap



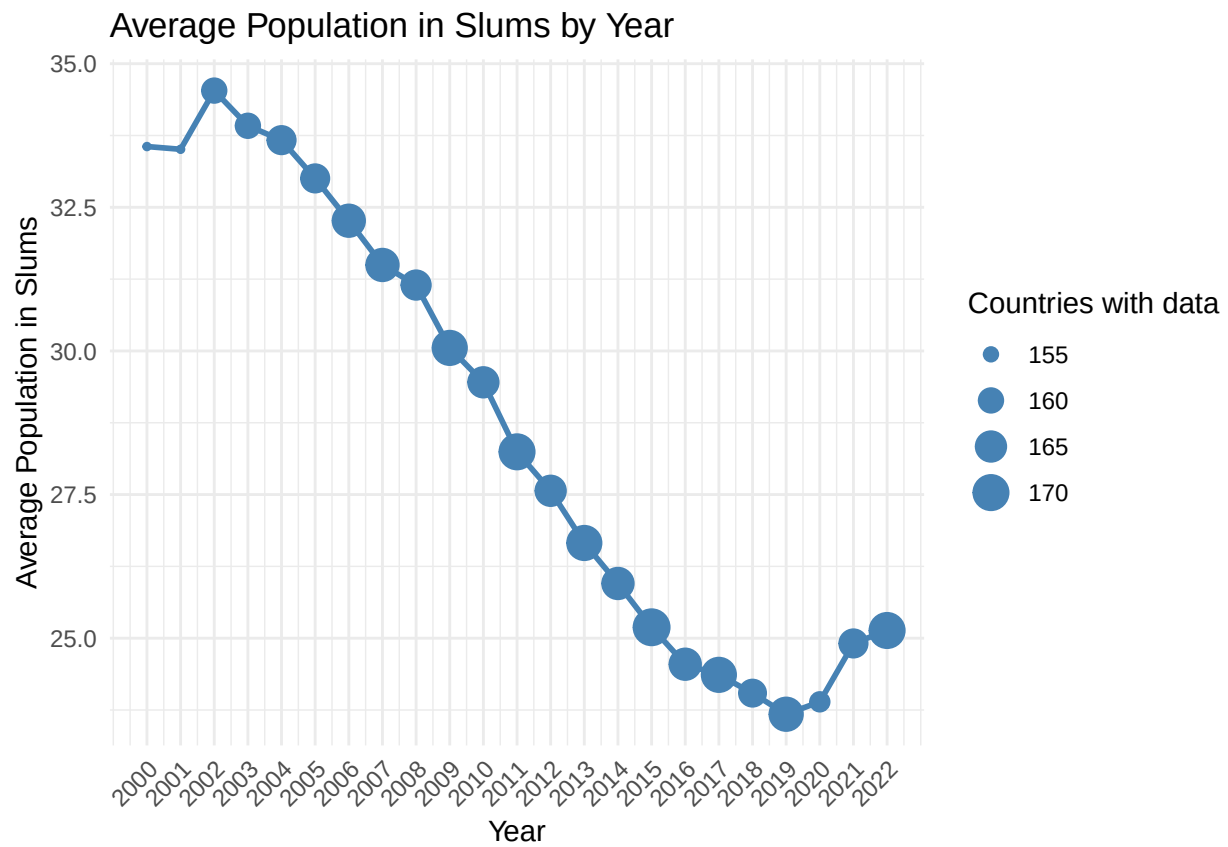
```
#
# Now we will tackle PopInSlums with straightforward interpolation of missing
# years, which follow an every-other-year pattern except for 2009 and 2019 which
# have a small handful of data points

# Plot PopInSLums by year
PlotSlums(PreProcessingData)
```

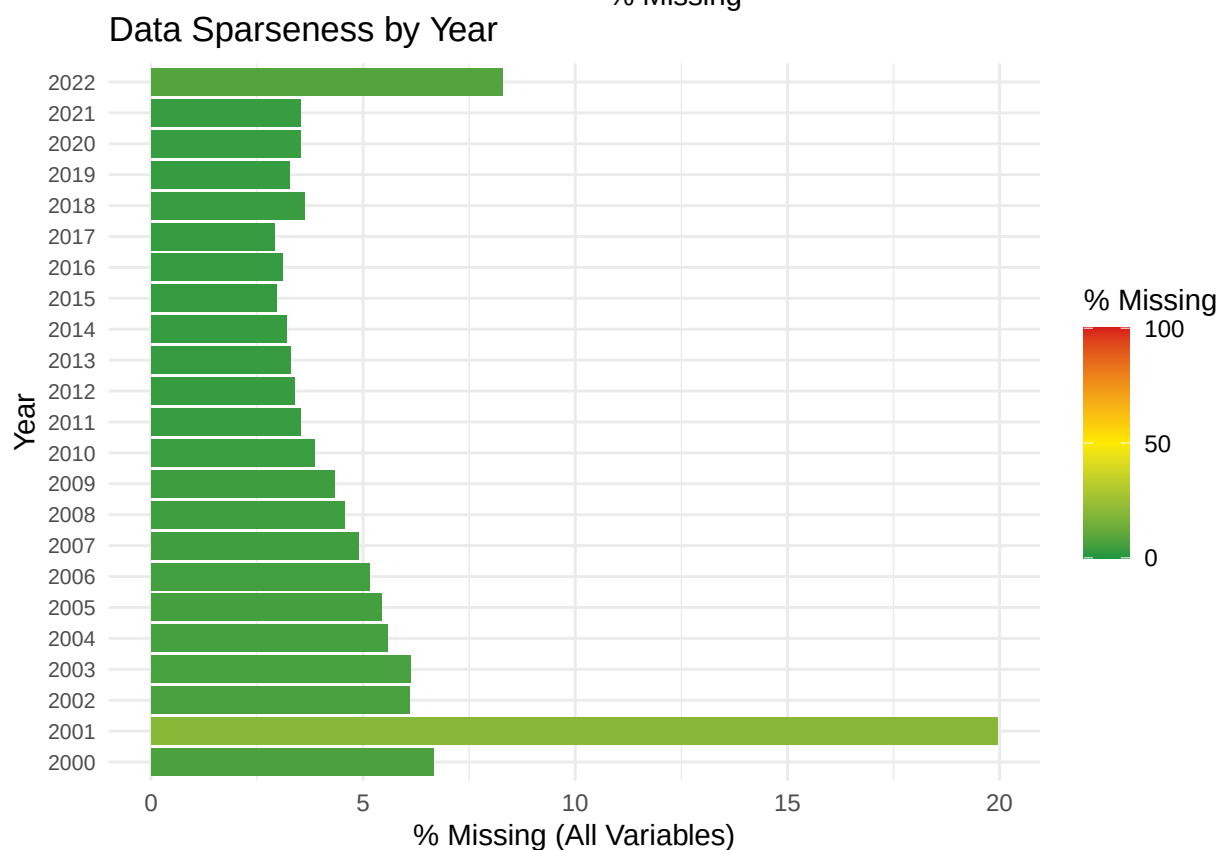
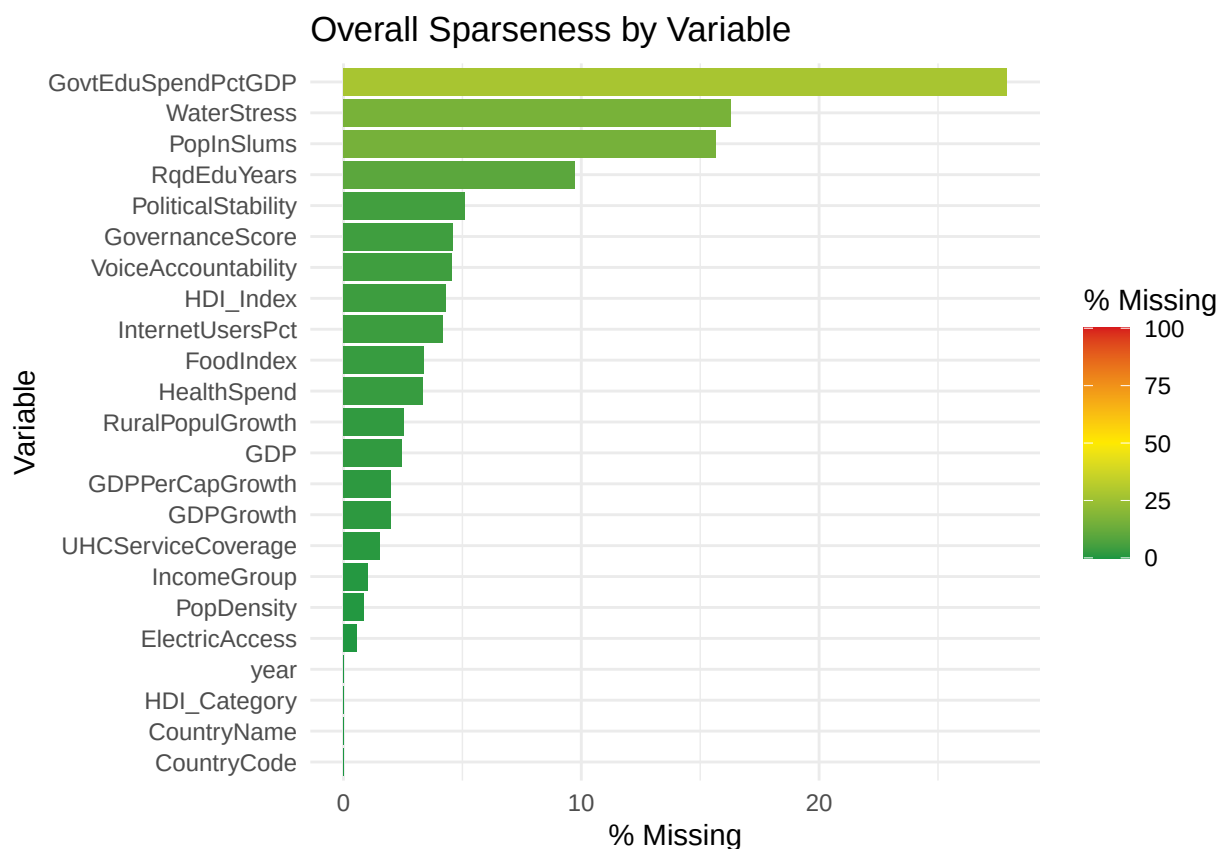


```
# Linearly interpolate PopInSlums
PreProcessingData <- PreProcessingData %>%
  group_by(CountryCode) %>%
  arrange(CountryCode, year) %>%
  mutate(PopInSlums = if_else(
    year %% 2 == 1 & is.na(PopInSlums), # Only odd years and NA
    na.approx(PopInSlums, x = year, na.rm = FALSE), # interpolate
    PopInSlums # keep existing values
  )) %>%
  ungroup()

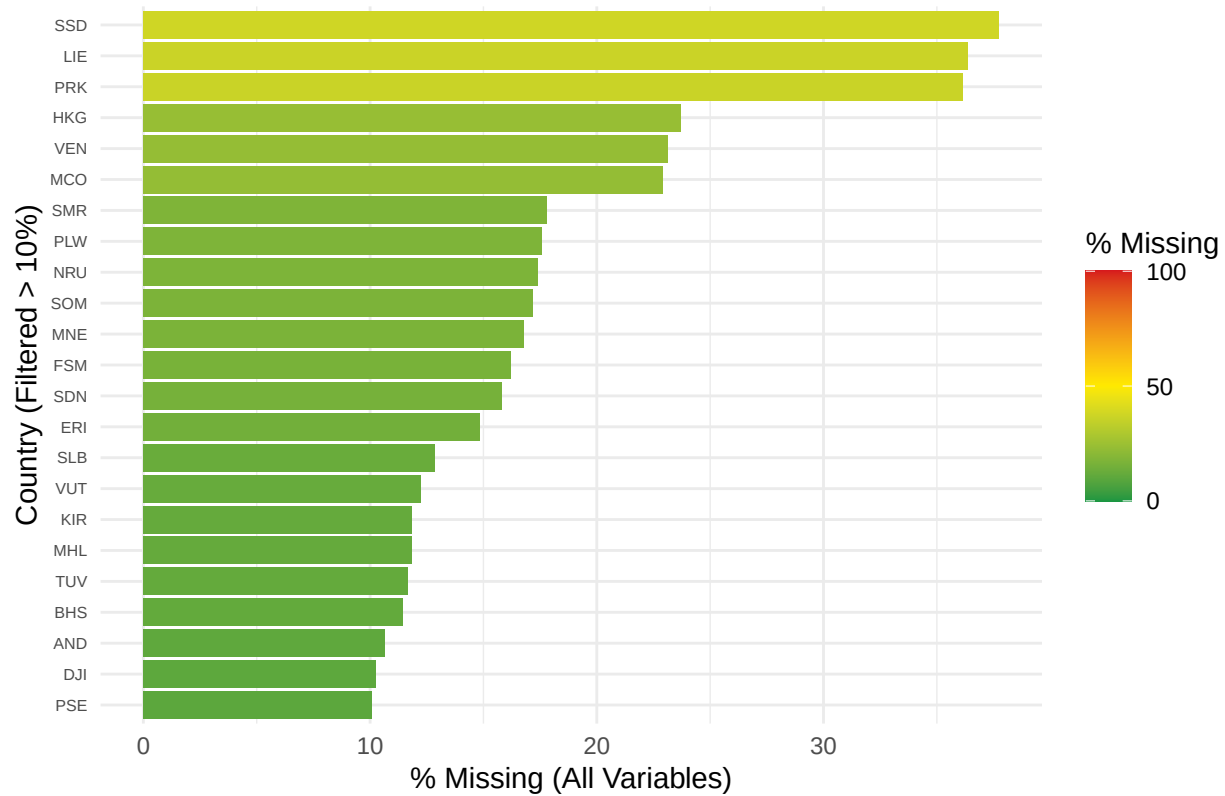
# Re-plot
PlotSlums(PreProcessingData)
```



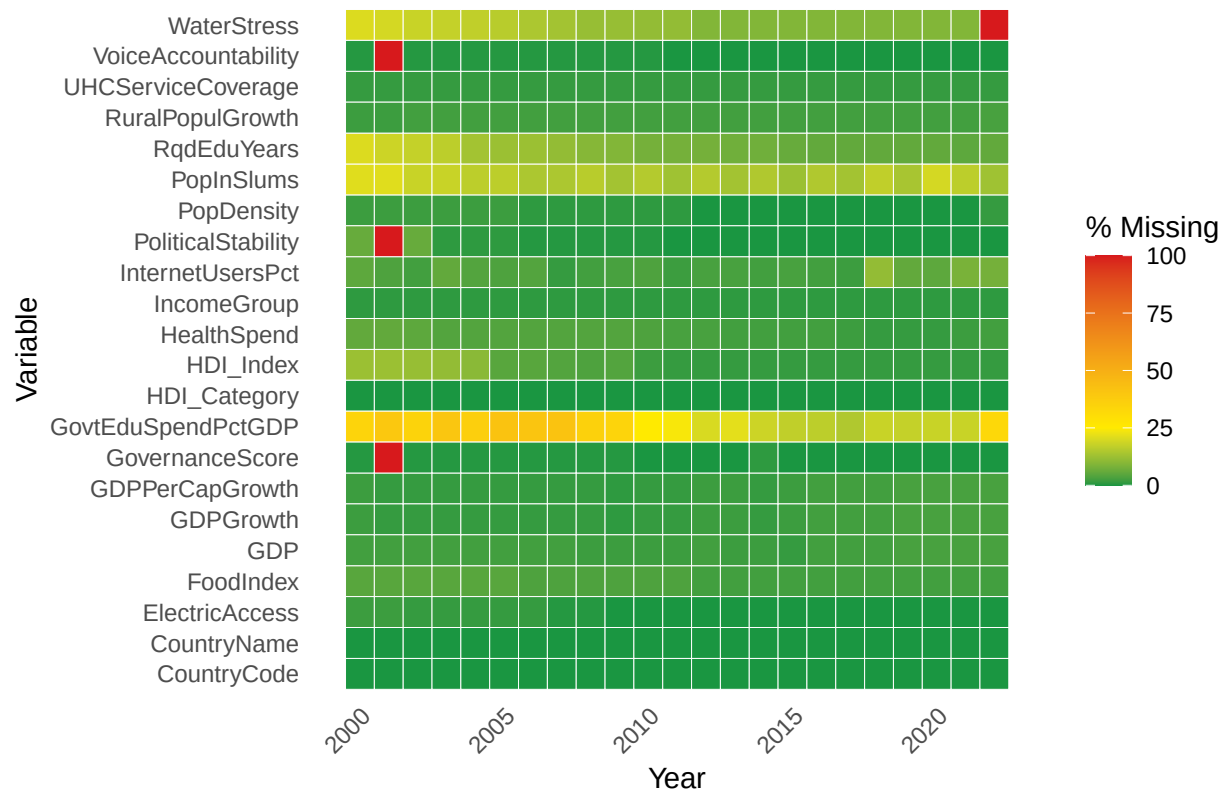
```
# Rerun all charts.  
FourPlots(PreProcessingData, 10)
```



Data Sparseness by Country (High-Missingness Only)



Missing Data Heatmap



```
# Now we interpolate the missing 2001 VoiceAccountability, PoliticalStability
# and GovernanceScore data
```

```
PreProcessingData %>%
  select(VoiceAccountability, PoliticalStability, GovernanceScore) %>%
  pivot_longer(everything(), names_to = "Variable", values_to = "Value") %>%
  group_by(Variable) %>%
  summarize(
    Min = min(Value, na.rm = TRUE),
    Max = max(Value, na.rm = TRUE),
    Mean = mean(Value, na.rm = TRUE),
    Median = median(Value, na.rm = TRUE),
    Count = sum(!is.na(Value))
  ) %>%
  arrange(Variable) %>%
  kable(digits = 3)
```

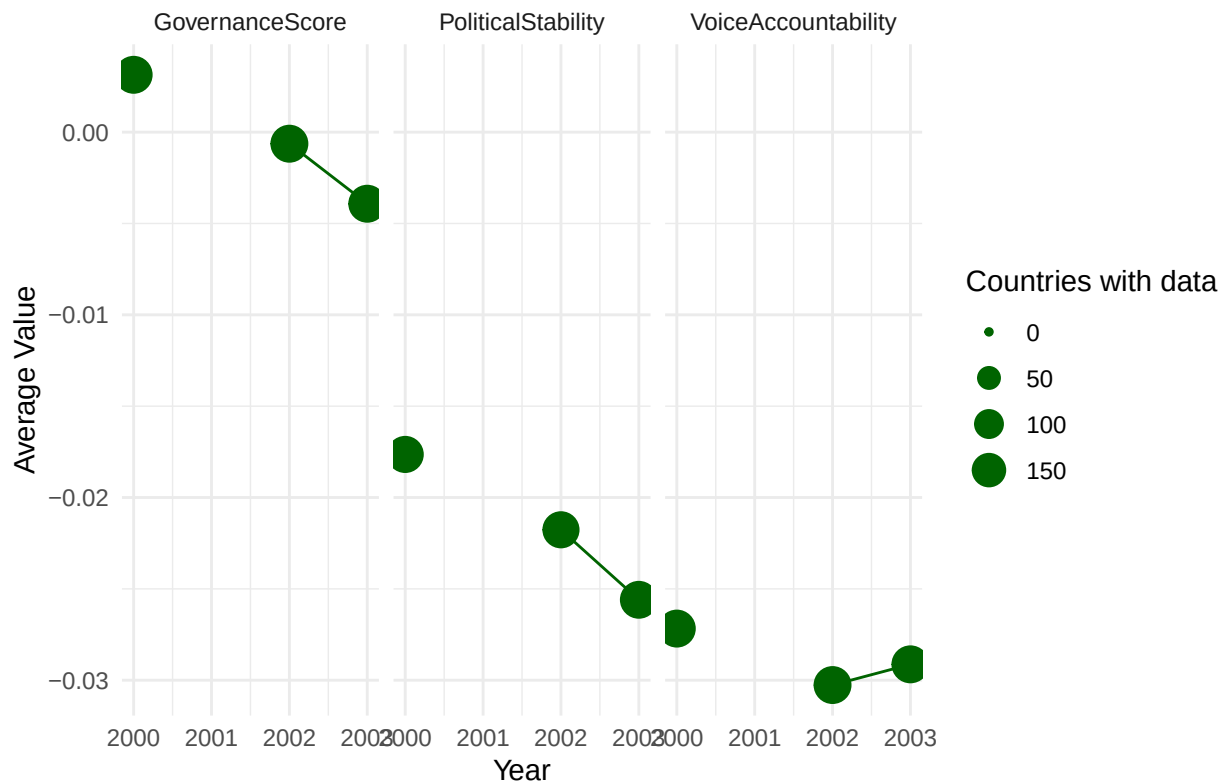
Variable	Min	Max	Mean	Median	Count
GovernanceScore	-2.245	2.259	-0.055	-0.231	4279
PoliticalStability	-3.313	1.759	-0.066	0.024	4255
VoiceAccountability	-2.313	1.801	-0.050	-0.005	4280

```
# Filter to years 2000-2003 and select the variables of interest
```

```
PreProcessingData %>%
  filter(year %in% 2000:2003) %>%
  select(CountryCode, year,
         VoiceAccountability, PoliticalStability, GovernanceScore) %>%
  pivot_longer(cols = -c(CountryCode, year), names_to = "Variable", values_to = "Value") %>%
  group_by(year, Variable) %>%
  summarize(
    mean_value = mean(Value, na.rm = TRUE),
    n_countries = sum(!is.na(Value)),
    .groups = "drop"
  ) %>%
  ggplot(aes(x = year, y = mean_value)) +
  geom_line(color = "darkgreen") +
  geom_point(aes(size = n_countries), color = "darkgreen") +
  facet_wrap(~ Variable, scales = "fixed") + # force same Y-axis for all plots
  # ylim(-0.1, 0.1) + # WGI scale range
  labs(
    title = "Average Values (2000-2003) for Key Governance Indicators",
    x = "Year",
    y = "Average Value",
    size = "Countries with data"
  ) +
  scale_x_continuous(breaks = 2000:2003) +
  theme_minimal()
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range
## (`geom_point()`).
```


Average Values (2000–2003) for Key Governance Indicators



```
gov_vars <- c("VoiceAccountability", "PoliticalStability", "GovernanceScore")

delta_check <- PreProcessingData %>%
  filter(year %in% c(2000, 2002, 2003)) %>%
  select(CountryCode, year, all_of(gov_vars)) %>%
  pivot_longer(cols = -c(CountryCode, year), names_to = "Variable", values_to = "Value") %>%
  pivot_wider(names_from = year, values_from = Value, names_prefix = "year_") %>%
  filter(!is.na(year_2000) & !is.na(year_2002) & !is.na(year_2003)) %>%
  mutate(
    slope_2000_2002 = (year_2002 - year_2000) / 2,
    slope_2002_2003 = (year_2003 - year_2002) / 1,
    slope_diff = abs(slope_2000_2002 - slope_2002_2003),
    changed_direction = sign(slope_2000_2002) != sign(slope_2002_2003)
  )

delta_check %>%
  group_by(Variable) %>%
  summarize(
    mean_slope_diff = mean(slope_diff),
    pct_large_deviation = mean(slope_diff > 0.25) * 100,
    max_deviation = max(slope_diff),
    n_chg_direction = sum(changed_direction),
    n = n()
  ) %>%
  arrange(desc(pct_large_deviation))
```

```
## # A tibble: 3 x 6
```

```
##   Variable      mean_slope_diff pct_large_deviation max_deviation n_chg_direction
##   <chr>          <dbl>          <dbl>          <dbl>          <int>
## 1 PoliticalSt~    0.314          49.2          1.37          105
## 2 VoiceAccoun~    0.125          10.8          0.573         103
## 3 GovernanceS~    0.101          8.76          1.01          94
## # i 1 more variable: n <int>
```

```
# Since there is a decent amount of deviation at the country level across 2020-2023,
# including that the slope of the curve changes direction more than 50% of the time,
# we are using a quadratic fit formula to estimate 2001 using data from 2000, 2002
# and 2003 - unless 2003 doesn't exist in which we case we interpolate 2000-2002
# or leave NA if insufficient data to do that
```

```
PreProcessingData <- PreProcessingData %>%
  arrange(CountryCode, year) %>%
  group_by(CountryCode) %>%
  group_modify(~{
    g <- .x
    for (v in gov_vars) {
      # only attempt if 2001 exists and is NA
      if (any(g$year == 2001) && is.na(g[[v]][g$year == 2001])) {
        # gather supporting points
        pts <- g %>% filter(year %in% c(2000, 2002, 2003)) %>%
          select(year, !!rlang::sym(v)) %>% stats::na.omit()

        if (nrow(pts) >= 3) {
          # quadratic fit through 2000, 2002, 2003
          mdl <- lm(formula = pts[[v]] ~ poly(pts$year, 2, raw = TRUE))
          pred <- predict(mdl, newdata = data.frame(year = 2001))[1]
          pred <- max(-2.5, min(2.5, pred)) # clamp to WGI range
          g[[v]][g$year == 2001] <- pred
        } else {
          # fallback: linear midpoint if 2000 and 2002 exist
          y0 <- g[[v]][g$year == 2000]
          y2 <- g[[v]][g$year == 2002]
          if (length(y0) == 1 && length(y2) == 1 && !is.na(y0) && !is.na(y2)) {
            pred <- (y0 + y2) / 2
            pred <- max(-2.5, min(2.5, pred))
            g[[v]][g$year == 2001] <- pred
          }
        }
      }
    }
    g
  }) %>%
  ungroup()
```

```
## Warning: 'newdata' had 1 row but variables found have 3 rows
## Warning: 'newdata' had 1 row but variables found have 3 rows
## Warning: 'newdata' had 1 row but variables found have 3 rows
## Warning: 'newdata' had 1 row but variables found have 3 rows
## Warning: 'newdata' had 1 row but variables found have 3 rows
## Warning: 'newdata' had 1 row but variables found have 3 rows
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

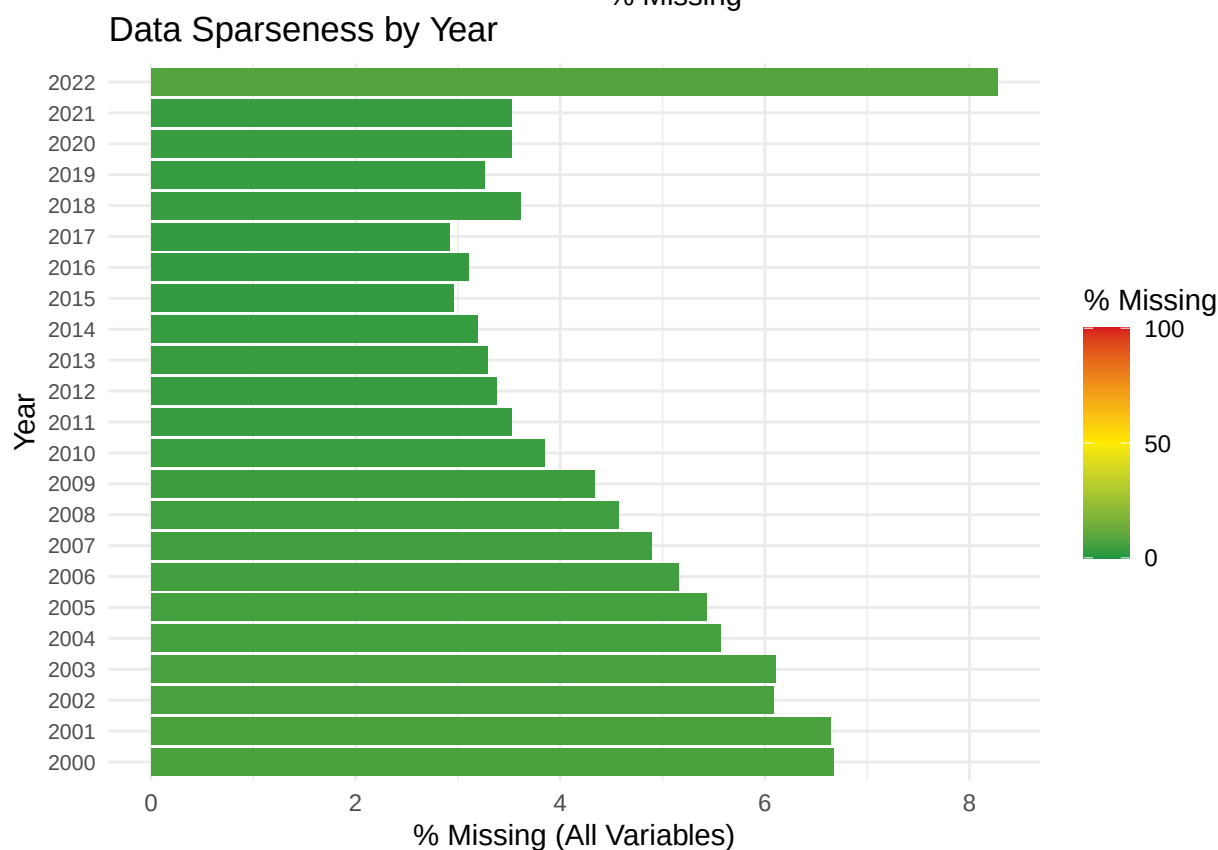
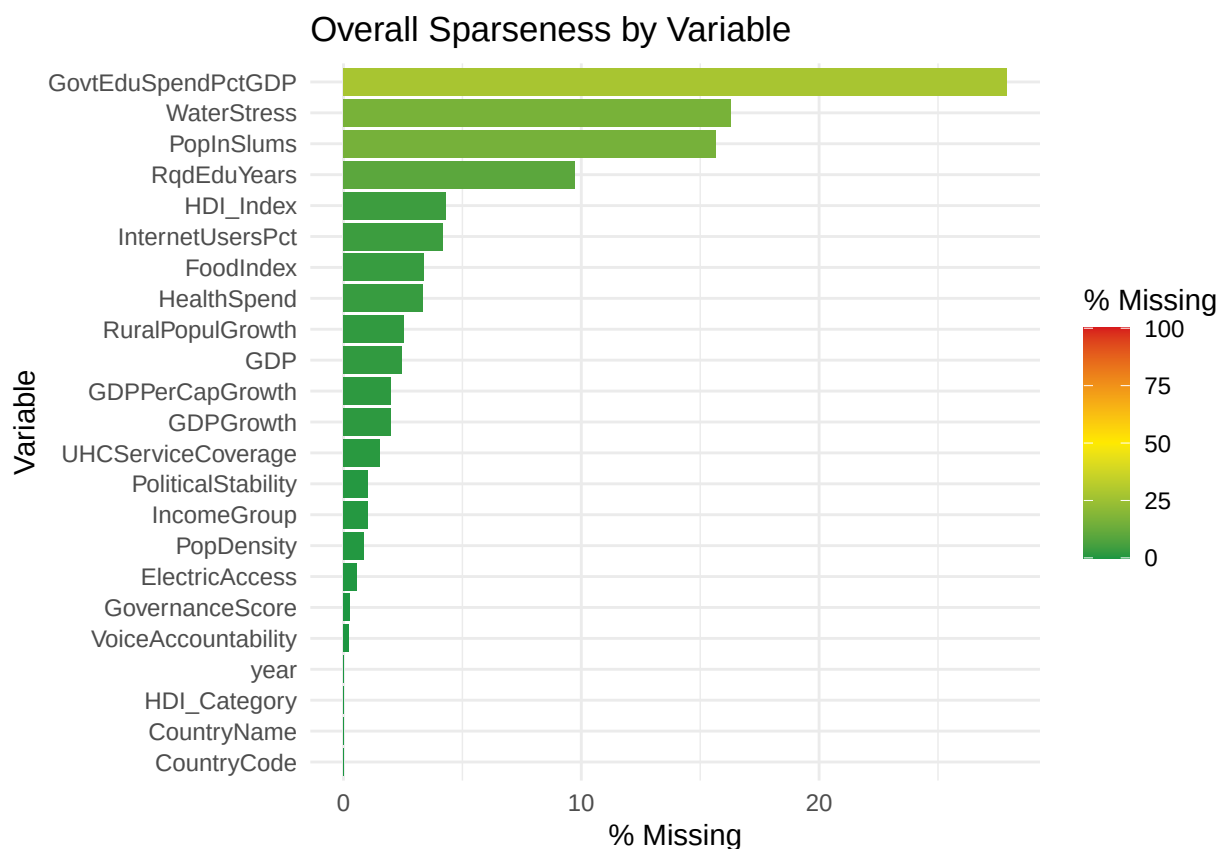
[illegible]

[illegible]

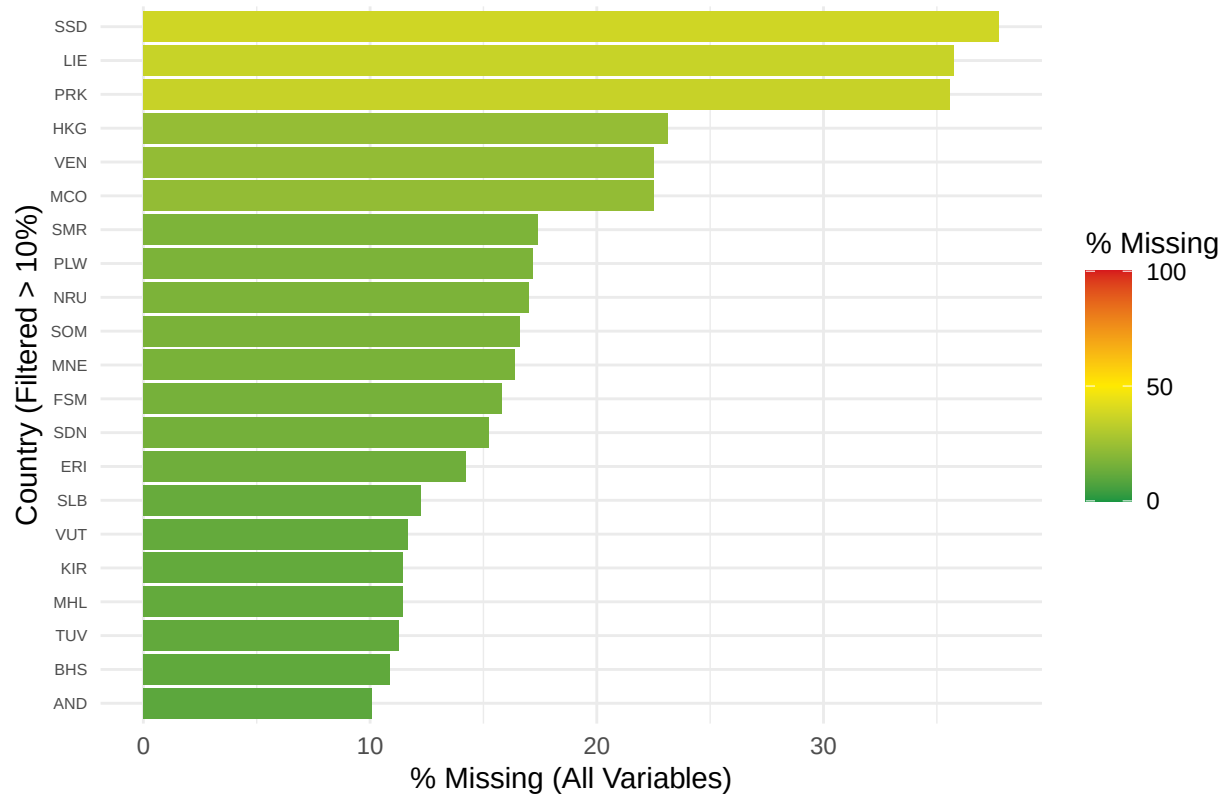
[illegible]

[illegible]

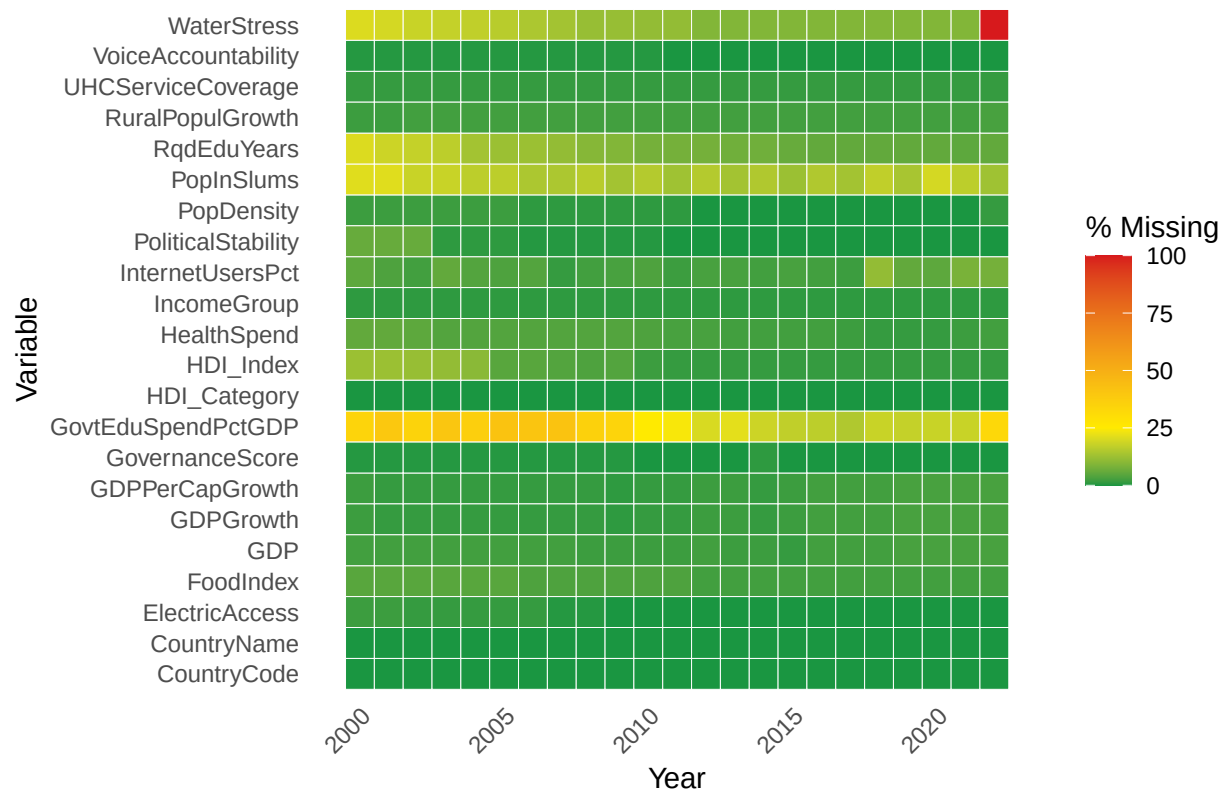
[illegible]



Data Sparseness by Country (High-Missingness Only)

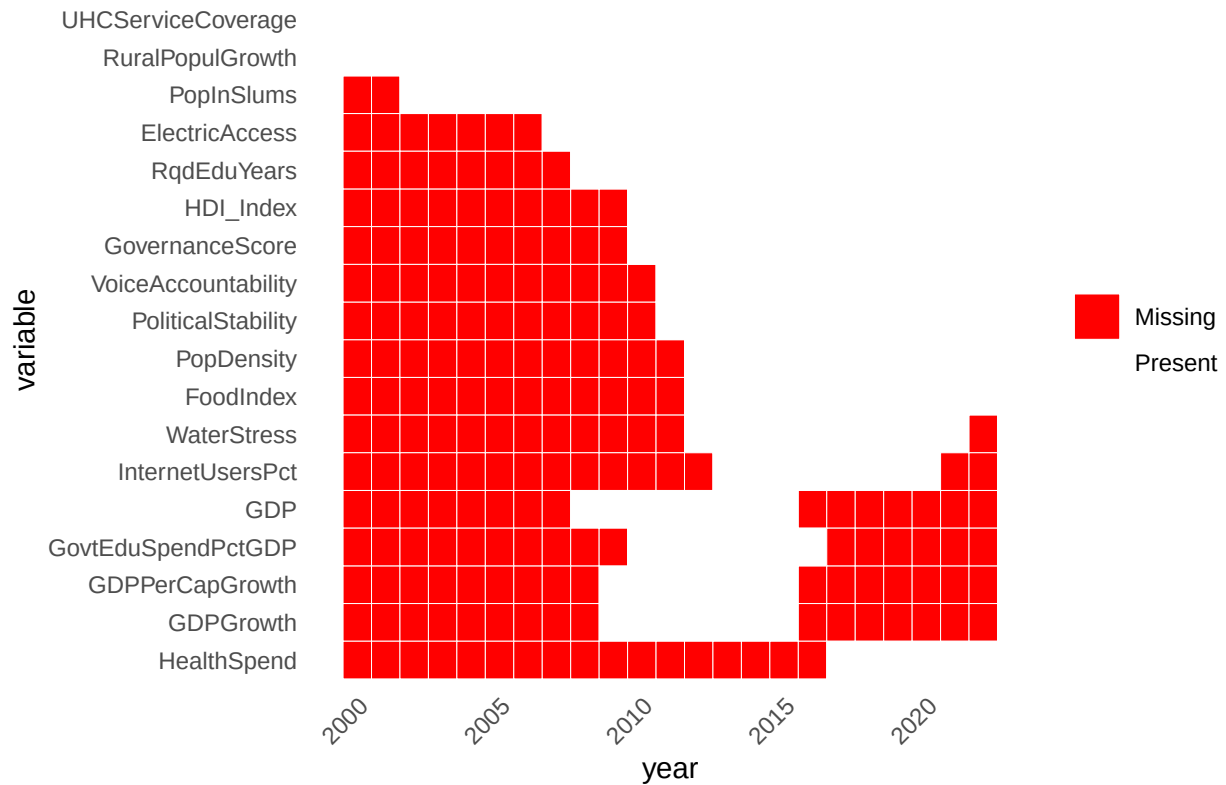


Missing Data Heatmap



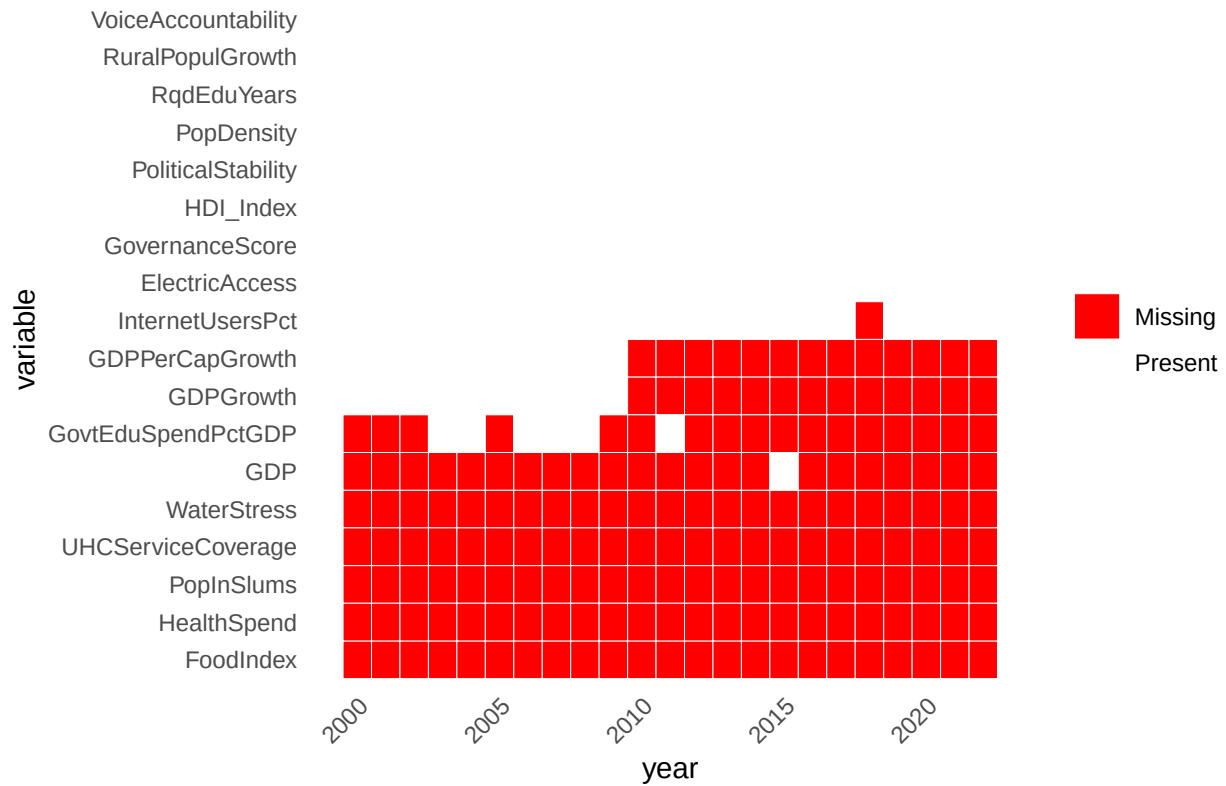
```
# Heatmap of missingness for the three countries over 20%
code <- "SSD"
OneCountryMissingness(PreProcessingData,code)
```

Data Availability Heatmap: SSD



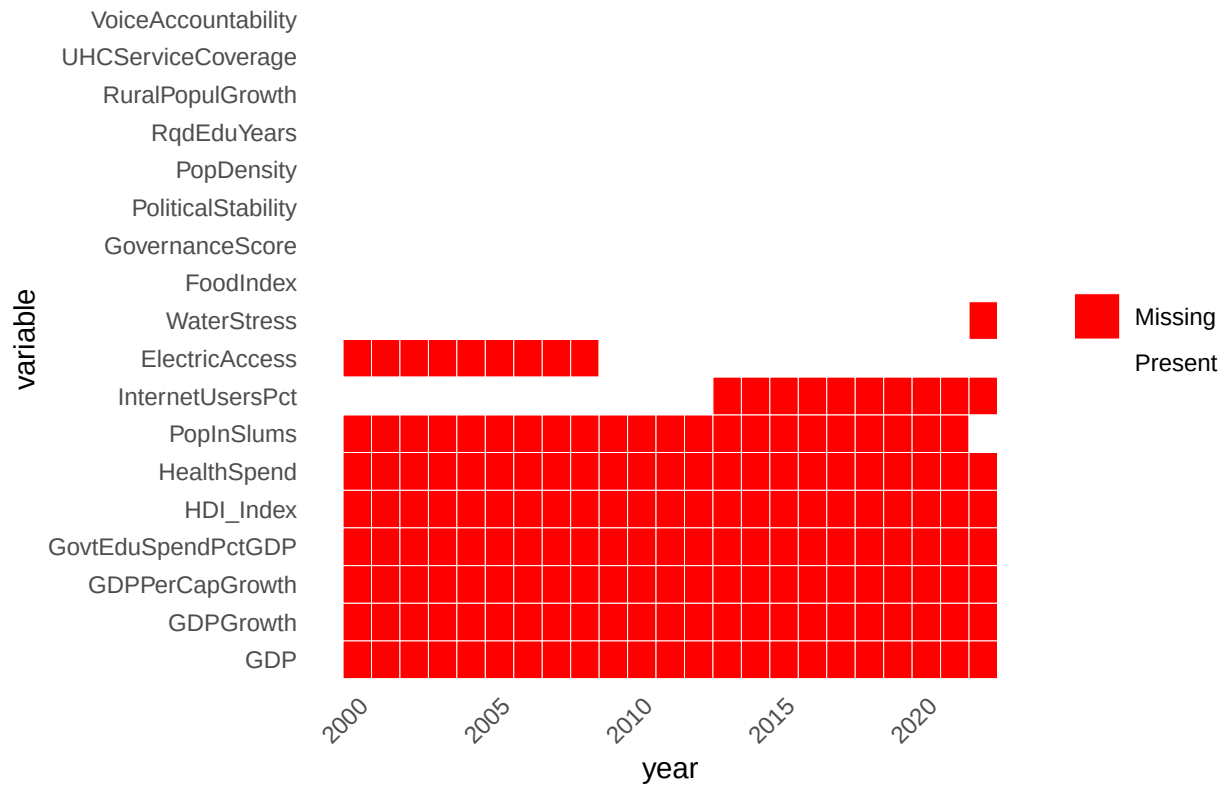
```
code <- "LIE"
OneCountryMissingness(PreProcessingData,code)
```


Data Availability Heatmap: LIE



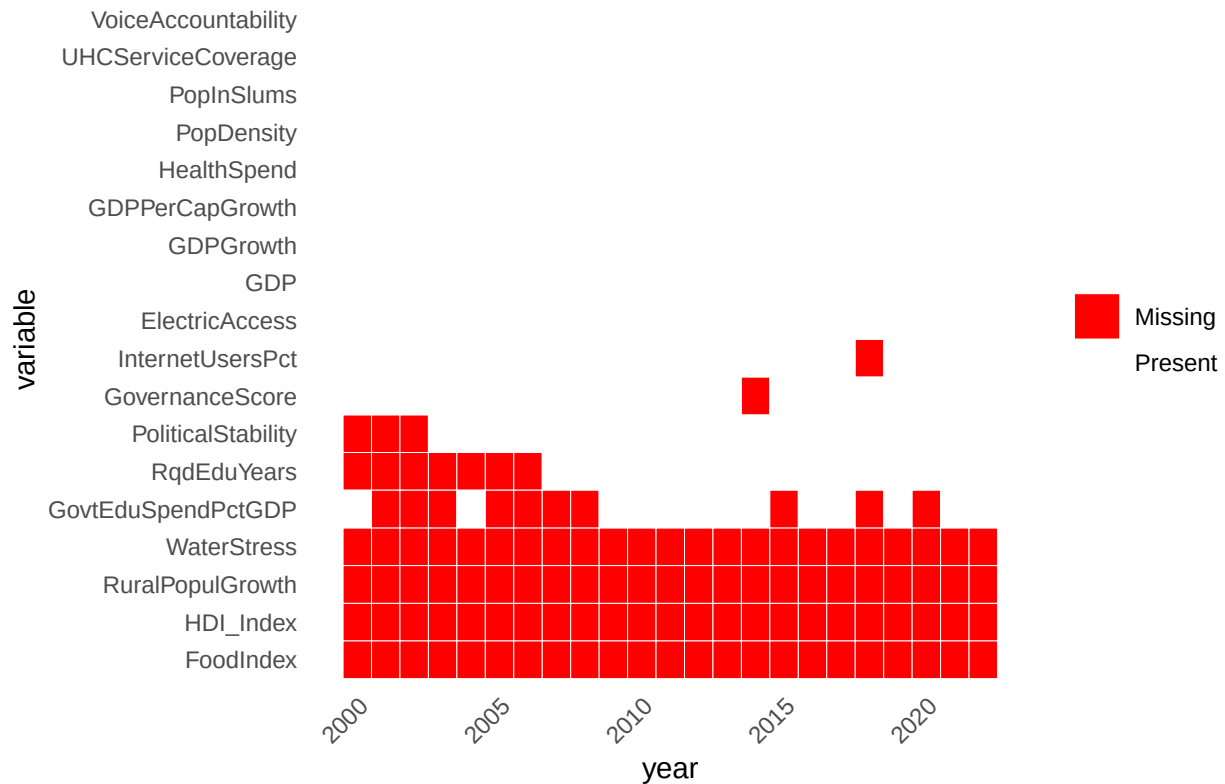
```
code <- "PRK"
OneCountryMissingness(PreProcessingData,code)
```

Data Availability Heatmap: PRK



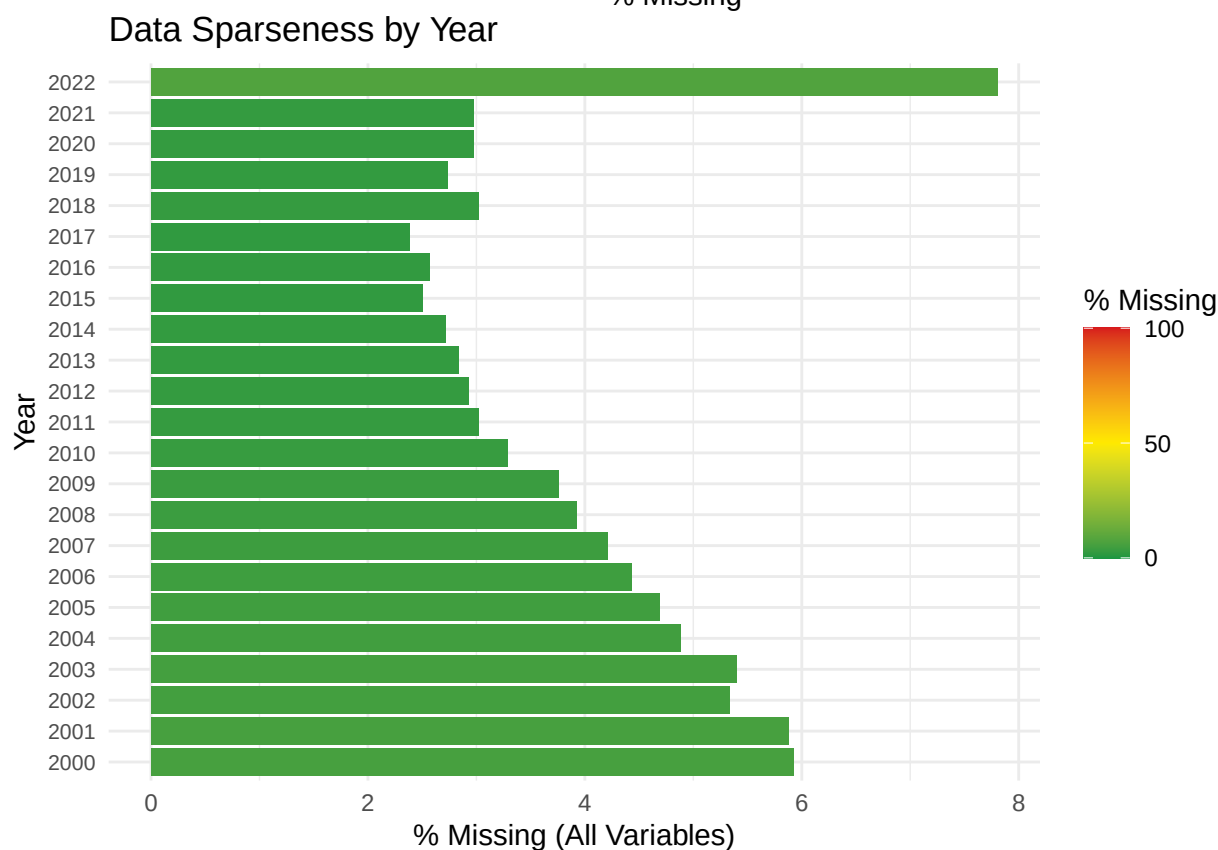
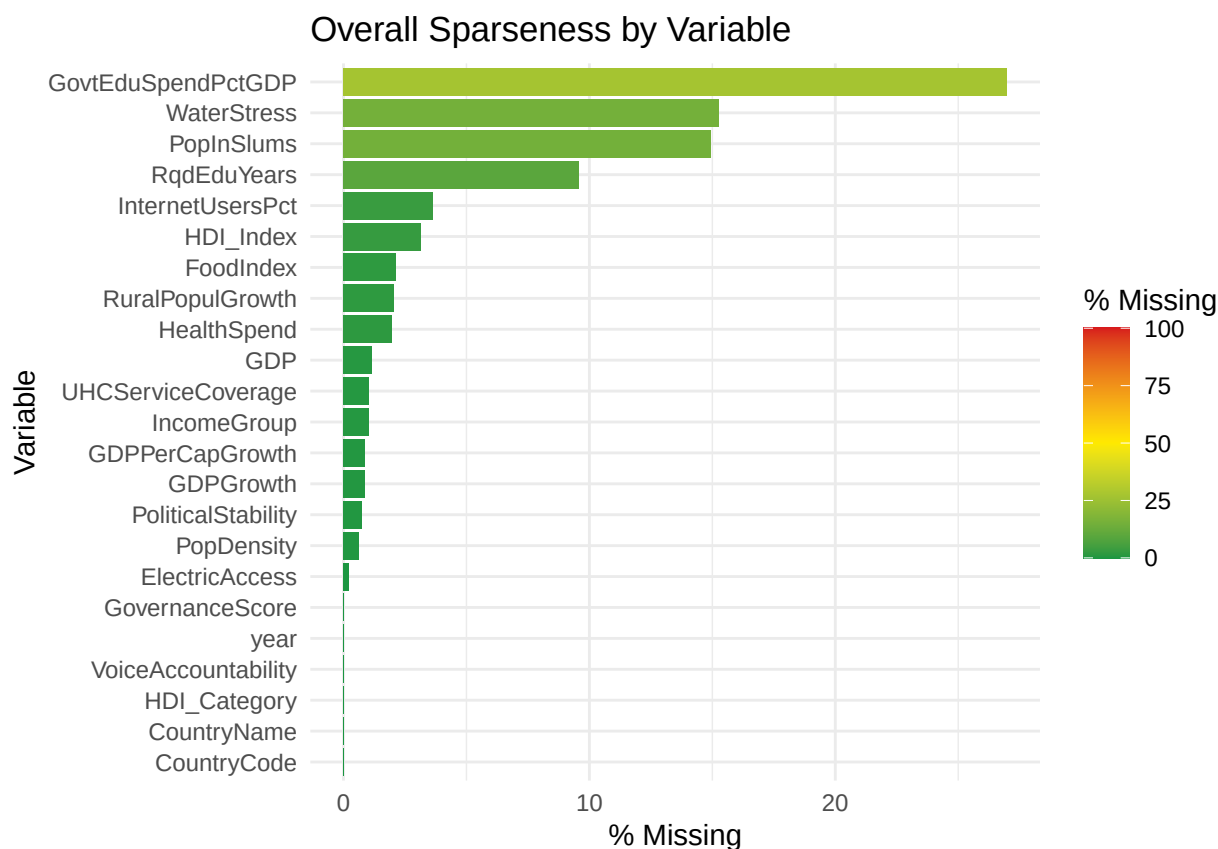
```
code <- "MCD"
OneCountryMissingness(PreProcessingData,code)
```

Data Availability Heatmap: MCO

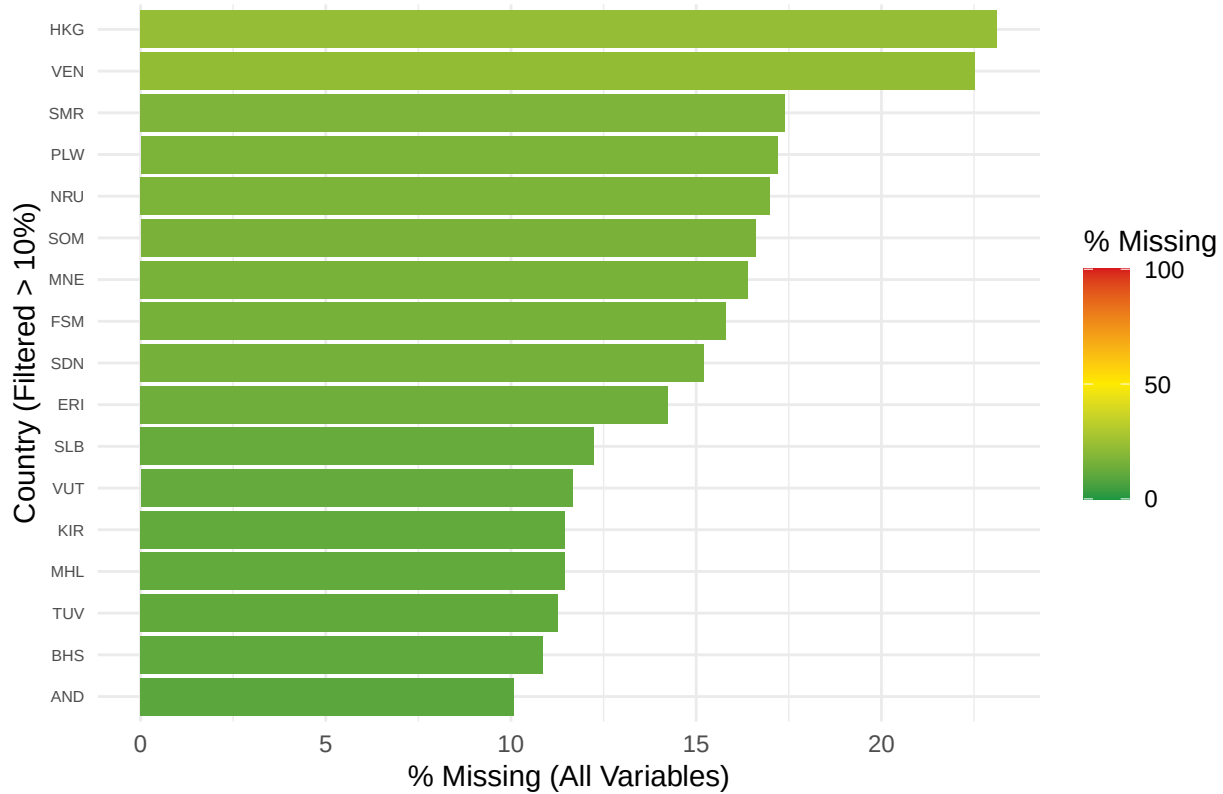


```
# Too much important data missing from all three so drop them entirely
# (Most would be dropped before modeling anyway)
PreProcessingData <- PreProcessingData %>%
  filter(!CountryCode %in% c("SSD", "LIE", "PRK", "MCO"))

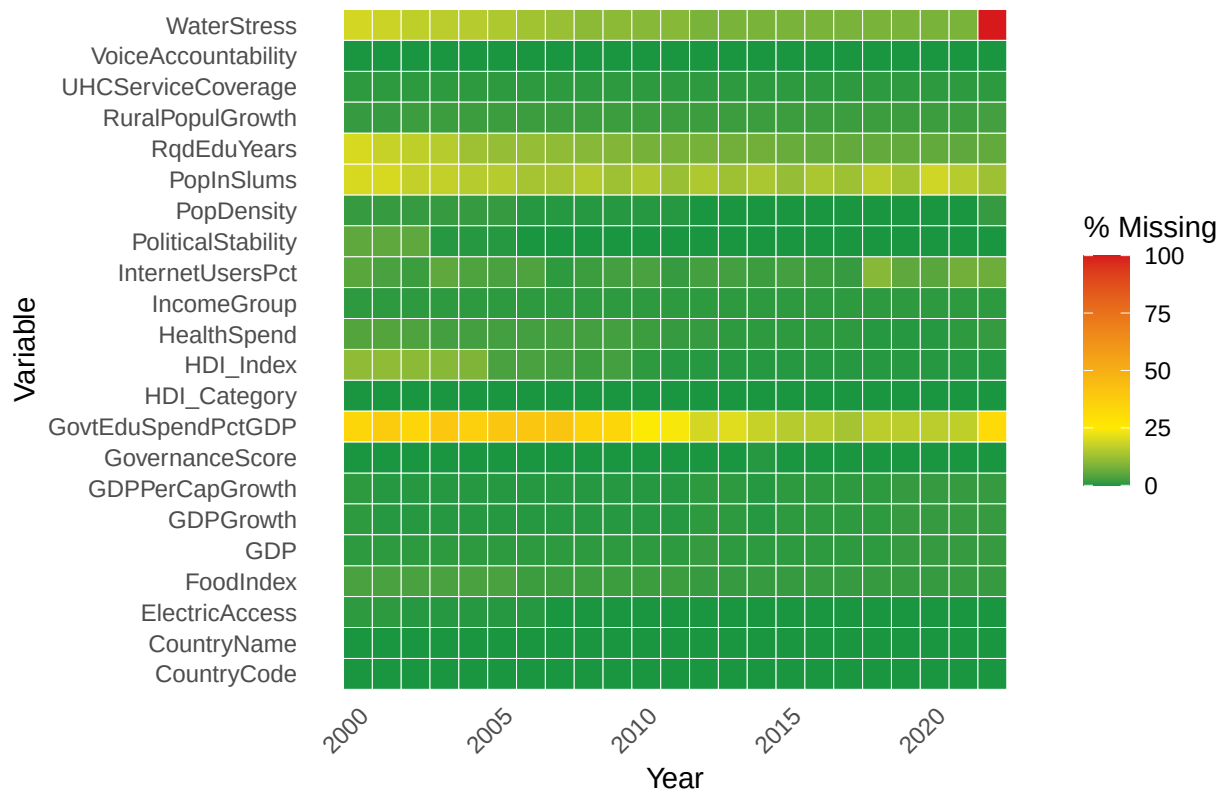
#Plot Again
FourPlots(PreProcessingData,10)
```



Data Sparseness by Country (High-Missingness Only)



Missing Data Heatmap



```

# Now fix the missing WaterStress data for 2022 via extrapolation

target_year <- 2022
lookback_n  <- 7
min_points  <- 3

print(paste0("Total Countries with data for 2022: ",
             sum(PreProcessingData$year == 2022)))

## [1] "Total Countries with data for 2022: 191"

print(paste0("Countries with NA for 2022 WaterStress: ",
             sum(PreProcessingData$year == 2022 &
                 is.na(PreProcessingData$WaterStress))))

## [1] "Countries with NA for 2022 WaterStress: 191"

print(paste0("Countries with at least 3 years of WaterStress data: ",
             sum(tapply(!is.na(PreProcessingData$WaterStress) &
                        PreProcessingData$year < 2022,
                        PreProcessingData$CountryCode, sum) >= 3)))

## [1] "Countries with at least 3 years of WaterStress data: 175"

hist_idx <- !is.na(PreProcessingData$WaterStress) & PreProcessingData$year < target_year
hist_df  <- PreProcessingData[hist_idx, c("CountryCode", "year", "WaterStress")]
hist_df  <- hist_df[!duplicated(hist_df[, c("CountryCode", "year")]), ]

## Split by country
by_ctype <- split(hist_df, hist_df$CountryCode)

## For each country, take recent lookback_n rows and fit lm; predict 2022 (or NA)
pred_vec <- sapply(names(by_ctype), function(cc) {
  df <- by_ctype[[cc]]
  if (is.null(df) || nrow(df) == 0) return(NA_real_)
  df <- df[order(df$year), ]
  if (nrow(df) > lookback_n) df <- df[(nrow(df) - lookback_n + 1):nrow(df), ]
  if (nrow(df) < min_points) return(NA_real_)
  fit <- try(lm(WaterStress ~ year, data = df), silent = TRUE)
  if (inherits(fit, "try-error")) return(NA_real_)
  p <- as.numeric(predict(fit, newdata = data.frame(year = target_year)))
  p
})

## Assemble predictions table (CountryCode, year=2022, pred)
pred_df <- data.frame(
  CountryCode = names(pred_vec),
  year        = target_year,
  pred        = as.numeric(pred_vec),
  row.names   = NULL,
  stringsAsFactors = FALSE
)

## Merge back to original; create WaterStress_imputed ONLY for 2022 missing rows
out <- merge(
  PreProcessingData,

```

```

    pred_df,
    by = c("CountryCode", "year"),
    all.x = TRUE,
    sort = FALSE
  )

  # If 2022 & WaterStress is NA & pred is not NA -> impute; else NA
  out$WaterStress_imputed <- ifelse(
    out$year == target_year & is.na(out$WaterStress) & !is.na(out$pred),
    out$pred,
    NA_real_
  )

  ## Drop helper column and return to original object name
  out$pred <- NULL
  PreProcessingData <- out

  imputed_2022 <- with(PreProcessingData,
    sum(year == 2022 & !is.na(WaterStress_imputed),
      na.rm = TRUE))
  imputed_2022

## [1] 175

# Show me the ones that are still NA
still_NA_2022 <- unique(PreProcessingData$CountryCode[
  PreProcessingData$year == 2022 & is.na(PreProcessingData$WaterStress_imputed)
])
still_NA_2022

## [1] "AND" "BHS" "FSM" "HKG" "KIR" "MHL" "MNE" "NRU" "PLW" "SLB" "SMR" "SYC"
## [13] "TON" "TUV" "WSM" "VUT"

# Countries (among the 13) with >=3 valid pre-2022 WaterStress values
eligible_13 <- names(which(tapply(
  PreProcessingData$year < 2022 & !is.na(PreProcessingData$WaterStress),
  PreProcessingData$CountryCode, sum) >= 3))

eligible_13 <- intersect(still_NA_2022, eligible_13)
eligible_13

## character(0)

subset(PreProcessingData,
  CountryCode %in% eligible_13 & year < 2022 ,
  select = c("CountryCode", "year", "WaterStress"))

## [1] CountryCode year      WaterStress
## <0 rows> (or 0-length row.names)

ineligible_13 <- setdiff(still_NA_2022, eligible_13)
ineligible_13

## [1] "AND" "BHS" "FSM" "HKG" "KIR" "MHL" "MNE" "NRU" "PLW" "SLB" "SMR" "SYC"
## [13] "TON" "TUV" "WSM" "VUT"

```

```
subset(PreProcessingData,
       CountryCode %in% ineligible_13 & year < 2022 ,
       select = c("CountryCode","year","WaterStress"))
```

##	CountryCode	year	WaterStress
## 245	AND	2000	NaN
## 246	AND	2001	NaN
## 247	AND	2002	NaN
## 248	AND	2003	NaN
## 249	AND	2004	NaN
## 250	AND	2005	NaN
## 251	AND	2006	NaN
## 252	AND	2007	NaN
## 253	AND	2008	NaN
## 254	AND	2009	NaN
## 255	AND	2010	NaN
## 256	AND	2011	NaN
## 257	AND	2012	NaN
## 258	AND	2013	NaN
## 259	AND	2014	NaN
## 260	AND	2015	NaN
## 261	AND	2016	NaN
## 262	AND	2017	NaN
## 263	AND	2018	NaN
## 264	AND	2019	NaN
## 265	AND	2020	NaN
## 266	AND	2021	NaN
## 451	FSM	2011	NaN
## 590	BHS	2000	NaN
## 591	BHS	2001	NaN
## 592	BHS	2002	NaN
## 593	BHS	2003	NaN
## 594	BHS	2004	NaN
## 595	BHS	2005	NaN
## 596	BHS	2006	NaN
## 597	BHS	2007	NaN
## 598	BHS	2008	NaN
## 599	BHS	2009	NaN
## 600	BHS	2010	NaN
## 601	BHS	2011	NaN
## 602	BHS	2012	NaN
## 603	BHS	2013	NaN
## 604	BHS	2014	NaN
## 605	BHS	2015	NaN
## 606	BHS	2016	NaN
## 607	BHS	2017	NaN
## 608	BHS	2018	NaN
## 609	BHS	2019	NaN
## 610	BHS	2020	NaN
## 611	BHS	2021	NaN
## 750	HKG	2011	NaN
## 1210	KIR	2011	NaN
## 1533	FSM	2000	NaN
## 1534	FSM	2001	NaN

## 1535	FSM 2002	NaN
## 1536	FSM 2003	NaN
## 1537	FSM 2004	NaN
## 1538	FSM 2005	NaN
## 1539	FSM 2006	NaN
## 1540	FSM 2007	NaN
## 1541	FSM 2008	NaN
## 1542	FSM 2009	NaN
## 1543	FSM 2010	NaN
## 1545	FSM 2012	NaN
## 1546	FSM 2013	NaN
## 1547	FSM 2014	NaN
## 1548	FSM 2015	NaN
## 1549	FSM 2016	NaN
## 1550	FSM 2017	NaN
## 1551	FSM 2018	NaN
## 1552	FSM 2019	NaN
## 1553	FSM 2020	NaN
## 1554	FSM 2021	NaN
## 1636	MHL 2000	NaN
## 1647	MHL 2011	NaN
## 1751	MNE 2000	NaN
## 1762	MNE 2011	NaN
## 1832	HKG 2000	NaN
## 1833	HKG 2001	NaN
## 1834	HKG 2002	NaN
## 1835	HKG 2003	NaN
## 1836	HKG 2004	NaN
## 1837	HKG 2005	NaN
## 1838	HKG 2006	NaN
## 1839	HKG 2007	NaN
## 1840	HKG 2008	NaN
## 1841	HKG 2009	NaN
## 1842	HKG 2010	NaN
## 1844	HKG 2012	NaN
## 1845	HKG 2013	NaN
## 1846	HKG 2014	NaN
## 1847	HKG 2015	NaN
## 1848	HKG 2016	NaN
## 1849	HKG 2017	NaN
## 1850	HKG 2018	NaN
## 1851	HKG 2019	NaN
## 1852	HKG 2020	NaN
## 1853	HKG 2021	NaN
## 2073	NRU 2000	NaN
## 2084	NRU 2011	NaN
## 2234	PLW 2000	NaN
## 2245	PLW 2011	NaN
## 2292	KIR 2000	NaN
## 2293	KIR 2001	NaN
## 2294	KIR 2002	NaN
## 2295	KIR 2003	NaN
## 2296	KIR 2004	NaN
## 2297	KIR 2005	NaN

## 2298	KIR 2006	NaN
## 2299	KIR 2007	NaN
## 2300	KIR 2008	NaN
## 2301	KIR 2009	NaN
## 2302	KIR 2010	NaN
## 2304	KIR 2012	NaN
## 2305	KIR 2013	NaN
## 2306	KIR 2014	NaN
## 2307	KIR 2015	NaN
## 2308	KIR 2016	NaN
## 2309	KIR 2017	NaN
## 2310	KIR 2018	NaN
## 2311	KIR 2019	NaN
## 2312	KIR 2020	NaN
## 2313	KIR 2021	NaN
## 2556	SLB 2000	NaN
## 2567	SLB 2011	NaN
## 2568	SLB 2012	NaN
## 2625	SMR 2000	NaN
## 2636	SMR 2011	NaN
## 2637	SMR 2012	NaN
## 2730	MHL 2001	NaN
## 2731	MHL 2002	NaN
## 2732	MHL 2003	NaN
## 2733	MHL 2004	NaN
## 2734	MHL 2005	NaN
## 2735	MHL 2006	NaN
## 2736	MHL 2007	NaN
## 2737	MHL 2008	NaN
## 2738	MHL 2009	NaN
## 2739	MHL 2010	NaN
## 2741	MHL 2012	NaN
## 2742	MHL 2013	NaN
## 2743	MHL 2014	NaN
## 2744	MHL 2015	NaN
## 2745	MHL 2016	NaN
## 2746	MHL 2017	NaN
## 2747	MHL 2018	NaN
## 2748	MHL 2019	NaN
## 2749	MHL 2020	NaN
## 2750	MHL 2021	NaN
## 2832	SYC 2000	NaN
## 2843	SYC 2011	NaN
## 2844	SYC 2012	NaN
## 2845	MNE 2001	NaN
## 2846	MNE 2002	NaN
## 2847	MNE 2003	NaN
## 2848	MNE 2004	NaN
## 2849	MNE 2005	NaN
## 2850	MNE 2006	NaN
## 2851	MNE 2007	NaN
## 2852	MNE 2008	NaN
## 2853	MNE 2009	NaN
## 2854	MNE 2010	NaN

## 2856	MNE 2012	NaN
## 2857	MNE 2013	NaN
## 2858	MNE 2014	NaN
## 2859	MNE 2015	NaN
## 2860	MNE 2016	NaN
## 2861	MNE 2017	NaN
## 2862	MNE 2018	NaN
## 2863	MNE 2019	NaN
## 2864	MNE 2020	NaN
## 2865	MNE 2021	NaN
## 3016	TON 2000	NaN
## 3027	TON 2011	NaN
## 3028	TON 2012	NaN
## 3108	TUV 2000	NaN
## 3119	TUV 2011	NaN
## 3120	TUV 2012	NaN
## 3167	NRU 2001	NaN
## 3168	NRU 2002	NaN
## 3169	NRU 2003	NaN
## 3170	NRU 2004	NaN
## 3171	NRU 2005	NaN
## 3172	NRU 2006	NaN
## 3173	NRU 2007	NaN
## 3174	NRU 2008	NaN
## 3175	NRU 2009	NaN
## 3176	NRU 2010	NaN
## 3178	NRU 2012	NaN
## 3179	NRU 2013	NaN
## 3180	NRU 2014	NaN
## 3181	NRU 2015	NaN
## 3182	NRU 2016	NaN
## 3183	NRU 2017	NaN
## 3184	NRU 2018	NaN
## 3185	NRU 2019	NaN
## 3186	NRU 2020	NaN
## 3187	NRU 2021	NaN
## 3328	PLW 2001	NaN
## 3329	PLW 2002	NaN
## 3330	PLW 2003	NaN
## 3331	PLW 2004	NaN
## 3332	PLW 2005	NaN
## 3333	PLW 2006	NaN
## 3334	PLW 2007	NaN
## 3335	PLW 2008	NaN
## 3336	PLW 2009	NaN
## 3337	PLW 2010	NaN
## 3338	VUT 2000	NaN
## 3339	PLW 2012	NaN
## 3340	PLW 2013	NaN
## 3341	PLW 2014	NaN
## 3342	PLW 2015	NaN
## 3343	PLW 2016	NaN
## 3344	PLW 2017	NaN
## 3345	PLW 2018	NaN

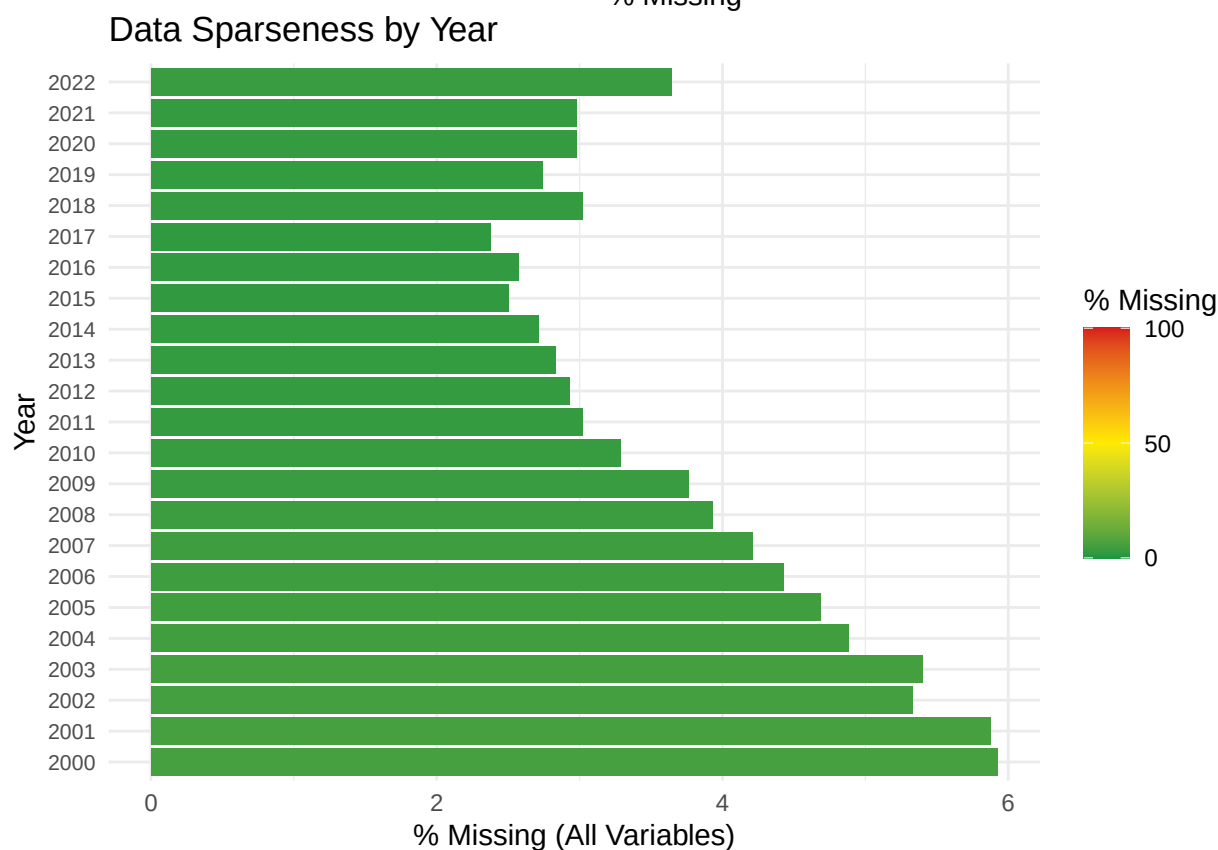
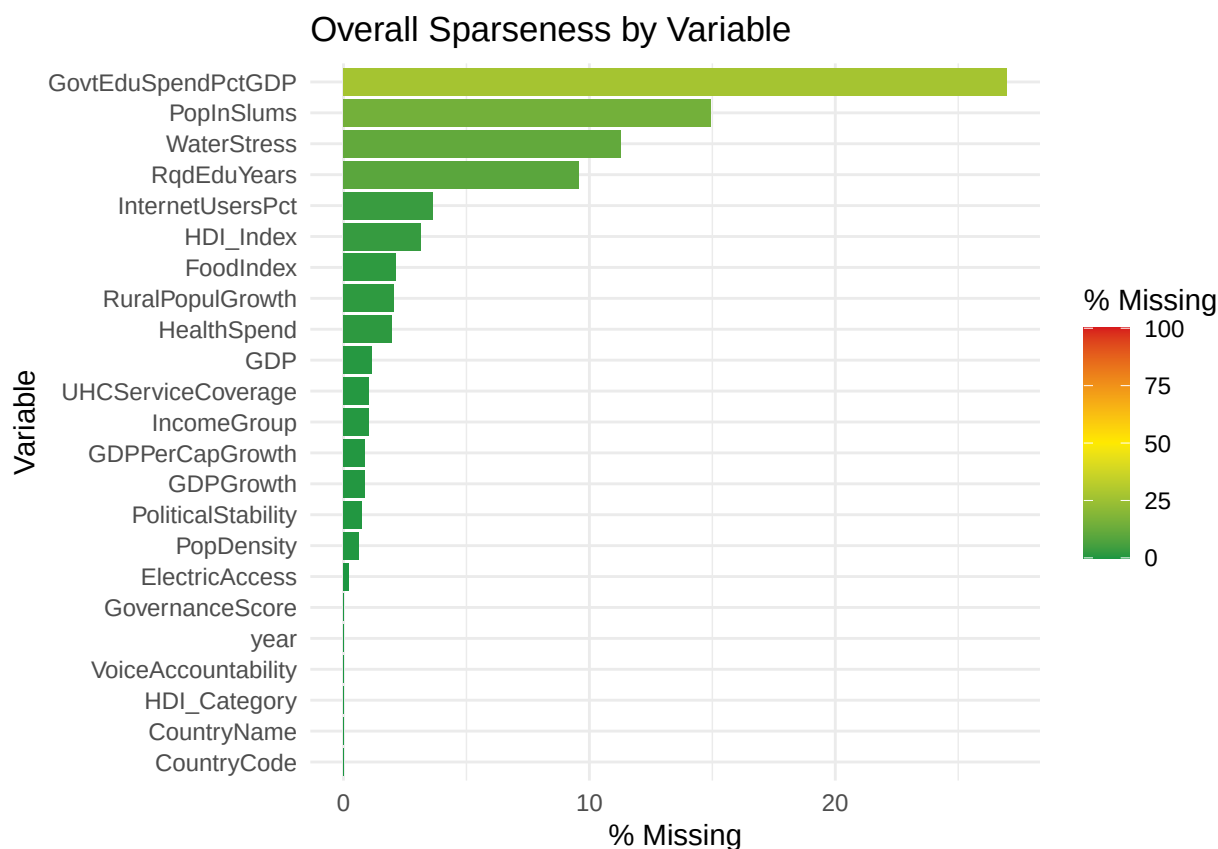
## 3346	PLW 2019	NaN
## 3347	PLW 2020	NaN
## 3348	PLW 2021	NaN
## 3350	VUT 2012	NaN
## 3361	WSM 2000	NaN
## 3372	WSM 2011	NaN
## 3373	WSM 2012	NaN
## 3568	SYC 2007	NaN
## 3569	SYC 2008	NaN
## 3579	SYC 2018	NaN
## 3580	SYC 2019	NaN
## 3650	SLB 2001	NaN
## 3651	SLB 2002	NaN
## 3652	SLB 2003	NaN
## 3653	SLB 2004	NaN
## 3654	SLB 2005	NaN
## 3655	SLB 2006	NaN
## 3656	SLB 2007	NaN
## 3657	SLB 2008	NaN
## 3658	SLB 2009	NaN
## 3659	SLB 2010	NaN
## 3662	SLB 2013	NaN
## 3663	SLB 2014	NaN
## 3664	SLB 2015	NaN
## 3665	SLB 2016	NaN
## 3666	SLB 2017	NaN
## 3667	SLB 2018	NaN
## 3668	SLB 2019	NaN
## 3669	SLB 2020	NaN
## 3670	SLB 2021	NaN
## 3719	SMR 2001	NaN
## 3720	SMR 2002	NaN
## 3721	SMR 2003	NaN
## 3722	SMR 2004	NaN
## 3723	SMR 2005	NaN
## 3724	SMR 2006	NaN
## 3725	SMR 2007	NaN
## 3726	SMR 2008	NaN
## 3727	SMR 2009	NaN
## 3728	SMR 2010	NaN
## 3731	SMR 2013	NaN
## 3732	SMR 2014	NaN
## 3733	SMR 2015	NaN
## 3734	SMR 2016	NaN
## 3735	SMR 2017	NaN
## 3736	SMR 2018	NaN
## 3737	SMR 2019	NaN
## 3738	SMR 2020	NaN
## 3739	SMR 2021	NaN
## 3752	TON 2007	NaN
## 3753	TON 2008	NaN
## 3763	TON 2018	NaN
## 3764	TON 2019	NaN
## 3840	TUV 2003	NaN

## 3841	TUV 2004	NaN
## 3845	TUV 2008	NaN
## 3851	TUV 2014	NaN
## 3852	TUV 2015	NaN
## 3855	TUV 2018	NaN
## 3856	TUV 2019	NaN
## 3926	SYC 2001	NaN
## 3927	SYC 2002	NaN
## 3928	SYC 2003	NaN
## 3929	SYC 2004	NaN
## 3930	SYC 2005	NaN
## 3931	SYC 2006	NaN
## 3934	SYC 2009	NaN
## 3935	SYC 2010	NaN
## 3938	SYC 2013	NaN
## 3939	SYC 2014	NaN
## 3940	SYC 2015	NaN
## 3941	SYC 2016	NaN
## 3942	SYC 2017	NaN
## 3945	SYC 2020	NaN
## 3946	SYC 2021	NaN
## 4070	VUT 2003	NaN
## 4071	VUT 2004	NaN
## 4074	VUT 2007	NaN
## 4075	VUT 2008	NaN
## 4081	VUT 2014	NaN
## 4082	VUT 2015	NaN
## 4085	VUT 2018	NaN
## 4086	VUT 2019	NaN
## 4093	WSM 2003	NaN
## 4094	WSM 2004	NaN
## 4097	WSM 2007	NaN
## 4098	WSM 2008	NaN
## 4108	WSM 2018	NaN
## 4109	WSM 2019	NaN
## 4110	TON 2001	NaN
## 4111	TON 2002	NaN
## 4112	TON 2003	NaN
## 4113	TON 2004	NaN
## 4114	TON 2005	NaN
## 4115	TON 2006	NaN
## 4118	TON 2009	NaN
## 4119	TON 2010	NaN
## 4122	TON 2013	NaN
## 4123	TON 2014	NaN
## 4124	TON 2015	NaN
## 4125	TON 2016	NaN
## 4126	TON 2017	NaN
## 4129	TON 2020	NaN
## 4130	TON 2021	NaN
## 4202	TUV 2001	NaN
## 4203	TUV 2002	NaN
## 4206	TUV 2005	NaN
## 4207	TUV 2006	NaN

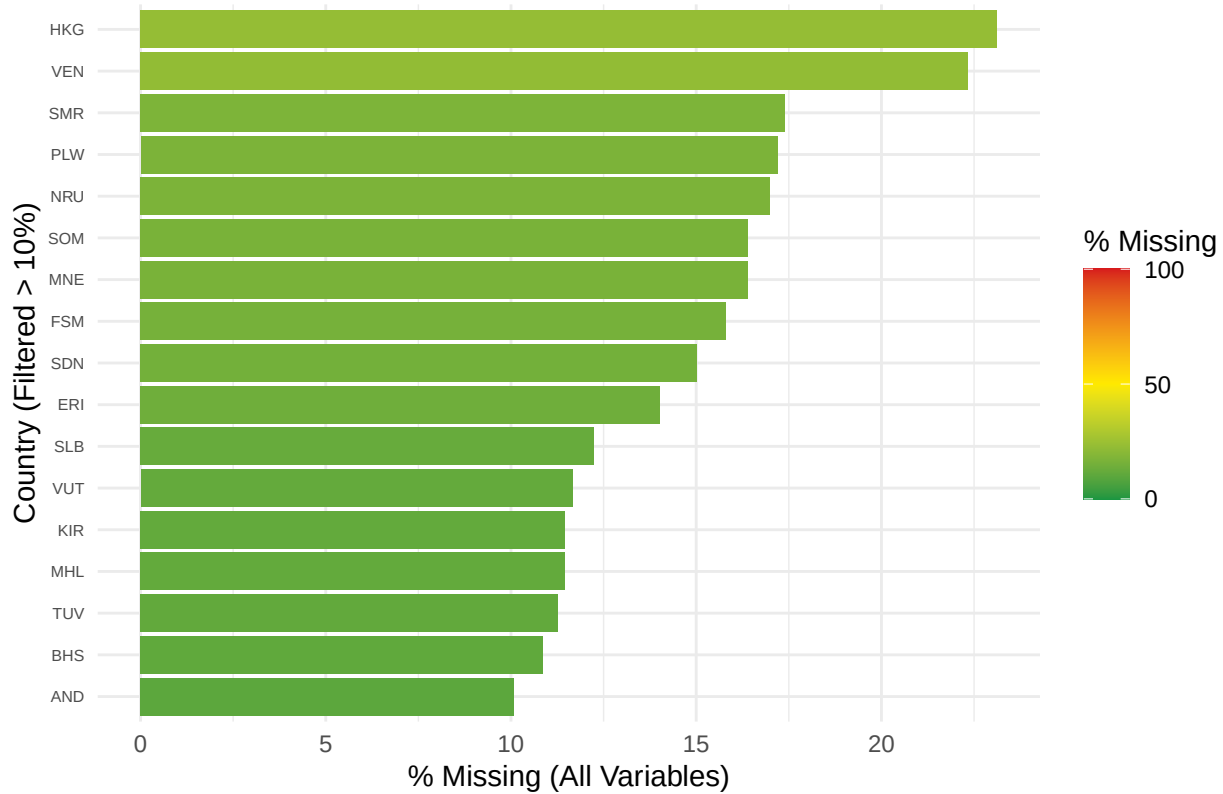
```
## 4208      TUV 2007      NaN
## 4209      VUT 2020      NaN
## 4210      TUV 2009      NaN
## 4211      TUV 2010      NaN
## 4213      WSM 2001      NaN
## 4214      TUV 2013      NaN
## 4217      TUV 2016      NaN
## 4218      TUV 2017      NaN
## 4221      TUV 2020      NaN
## 4222      TUV 2021      NaN
## 4232      WSM 2020      NaN
## 4312      VUT 2002      NaN
## 4315      VUT 2005      NaN
## 4316      VUT 2006      NaN
## 4319      VUT 2009      NaN
## 4320      VUT 2010      NaN
## 4326      VUT 2016      NaN
## 4327      VUT 2017      NaN
## 4330      WSM 2010      NaN
## 4331      VUT 2021      NaN
## 4335      WSM 2002      NaN
## 4338      WSM 2005      NaN
## 4339      WSM 2006      NaN
## 4342      WSM 2009      NaN
## 4346      WSM 2013      NaN
## 4347      WSM 2014      NaN
## 4348      WSM 2015      NaN
## 4349      WSM 2016      NaN
## 4350      WSM 2017      NaN
## 4354      WSM 2021      NaN
## 4366      VUT 2001      NaN
## 4386      VUT 2011      NaN
## 4387      VUT 2013      NaN
```

```
# Turns out there is NO data for any of those 13 countries
# Copy the imputed data over
# Copy 2022 imputed into WaterStress (only where WaterStress is NA), then drop helper column
idx <- PreProcessingData$year == 2022 & is.na(PreProcessingData$WaterStress) &
  !is.na(PreProcessingData$WaterStress_imputed)
PreProcessingData$WaterStress[idx] <- PreProcessingData$WaterStress_imputed[idx]
PreProcessingData$WaterStress_imputed <- NULL

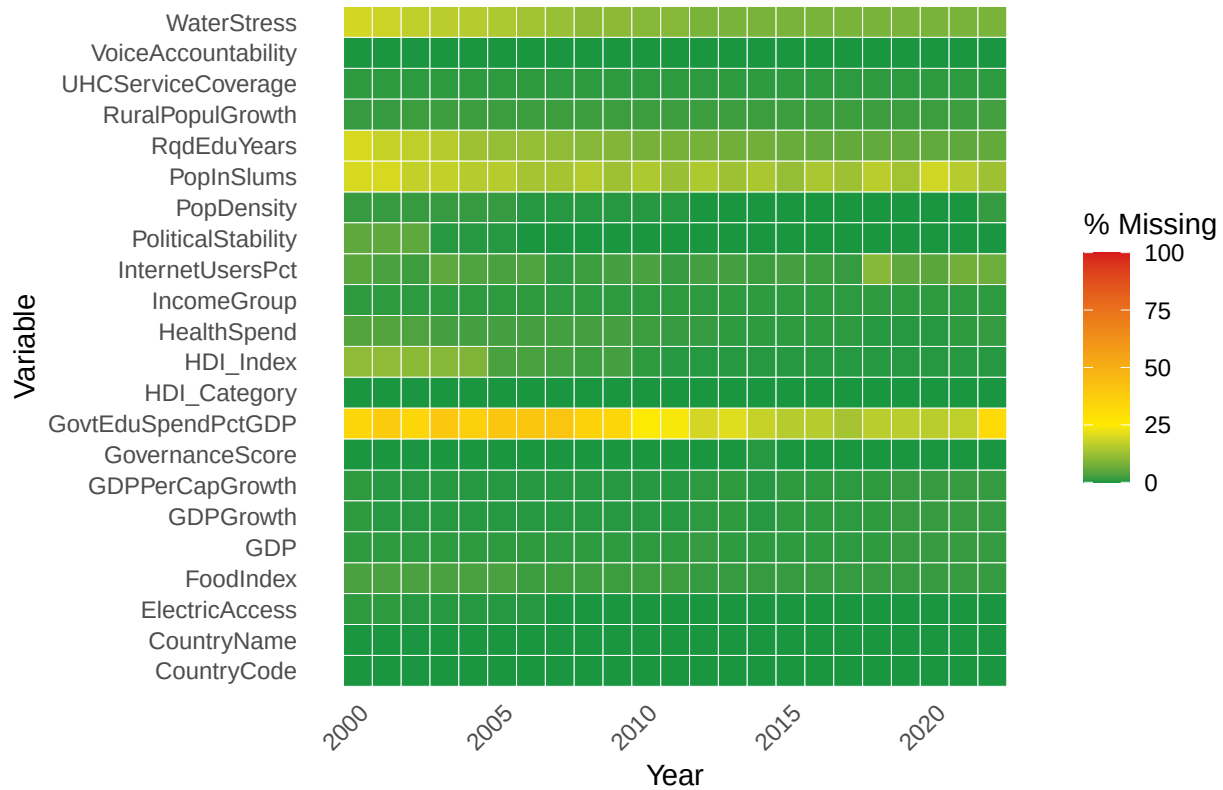
FourPlots(PreProcessingData,10)
```



Data Sparseness by Country (High-Missingness Only)



Missing Data Heatmap




```

# We are down to four features with material missing values. Let's see if
# we can just drop all rows with NA values first!

# Total number of rows
total_rows <- nrow(PreProcessingData)

# Number of rows with any NA
rows_with_na <- sum(!complete.cases(PreProcessingData))

# Percentage of rows that would be removed
percent_removed <- (rows_with_na / total_rows) * 100

# Number of rows remaining if you drop them
rows_remaining <- total_rows - rows_with_na

# Print results
cat("Rows with any NA:", rows_with_na, "\n")

## Rows with any NA: 2173

cat("Percent of rows removed:", round(percent_removed, 2), "%\n")

## Percent of rows removed: 49.47 %

cat("Rows remaining:", rows_remaining, "\n")

## Rows remaining: 2220

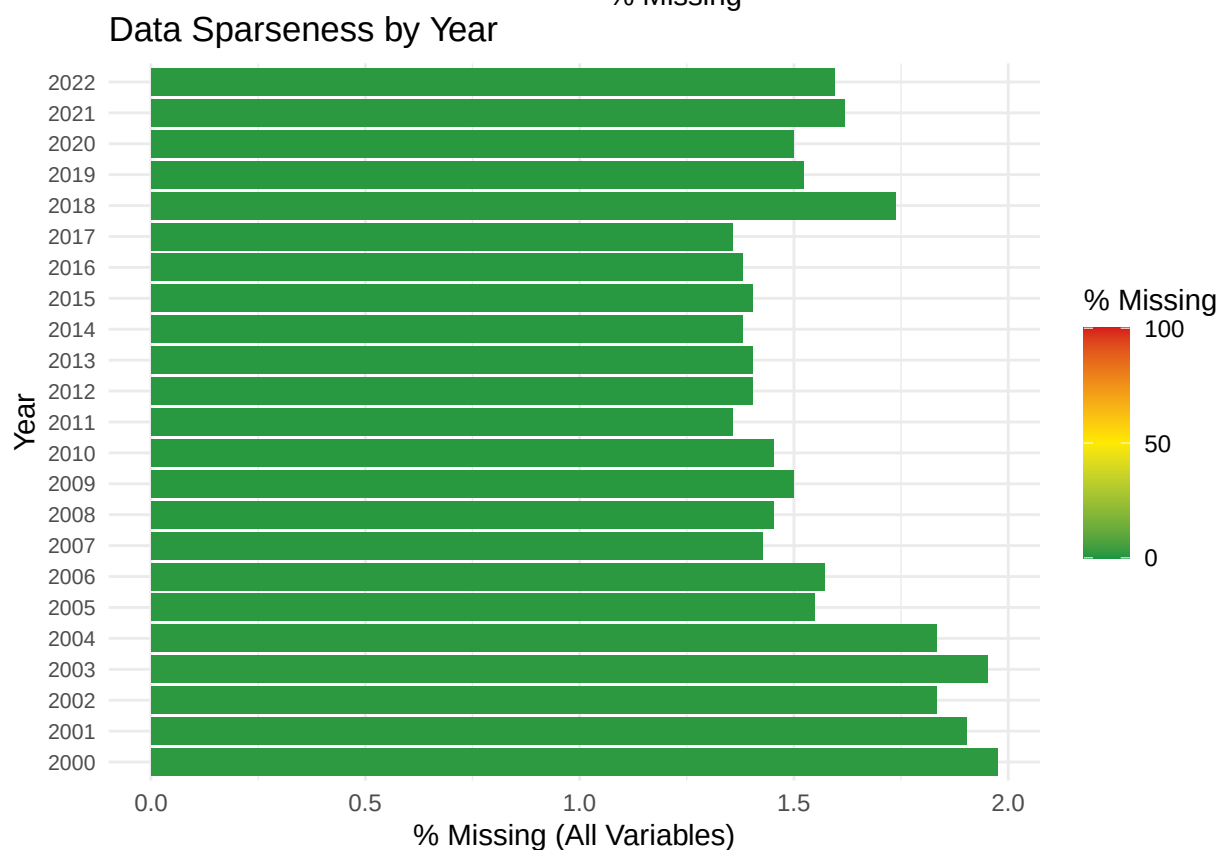
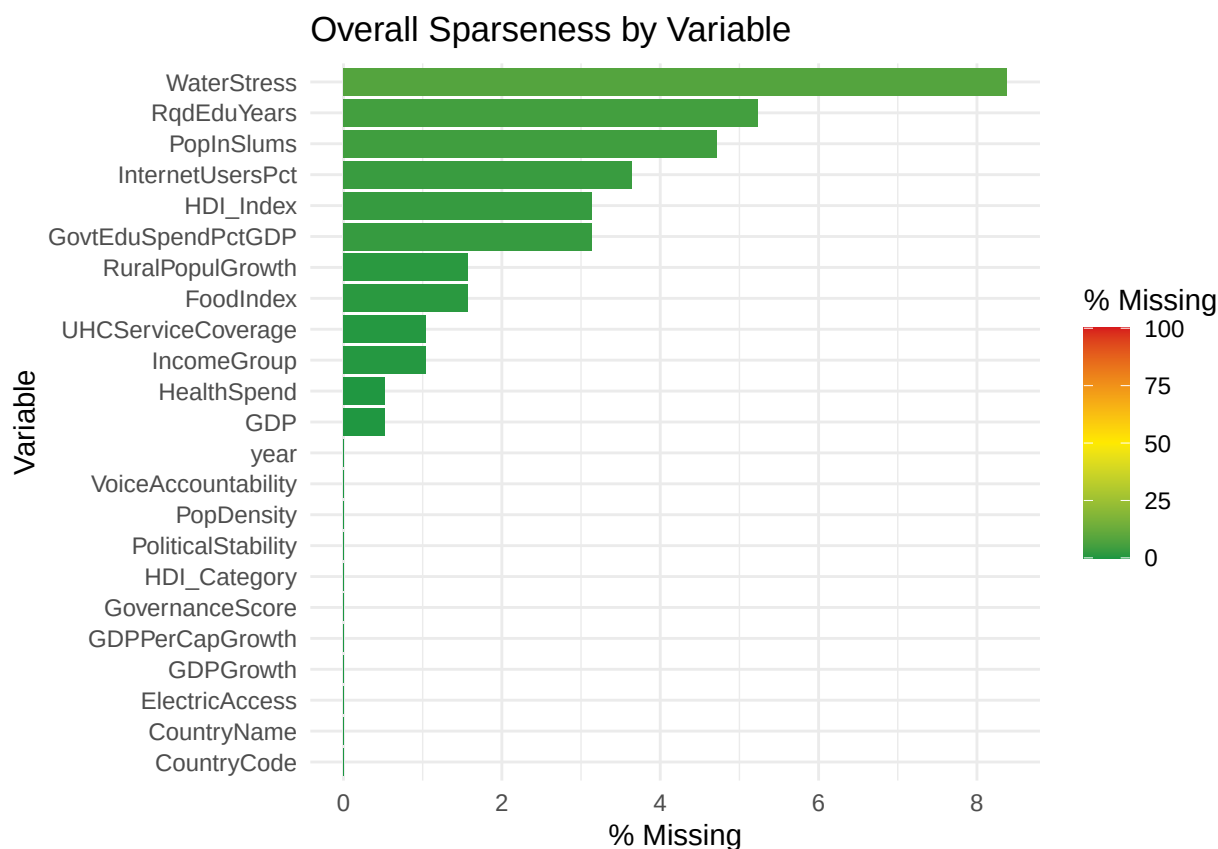
# Nope - we would lose too much of our data. So we will
# impute the missing data. That's fine for all but the top 4 variables, since
# the rest all have <5% missingness. We'll note it for the top 4 and run separate
# models that don't include them

# We use the impute_time_trend function to do this

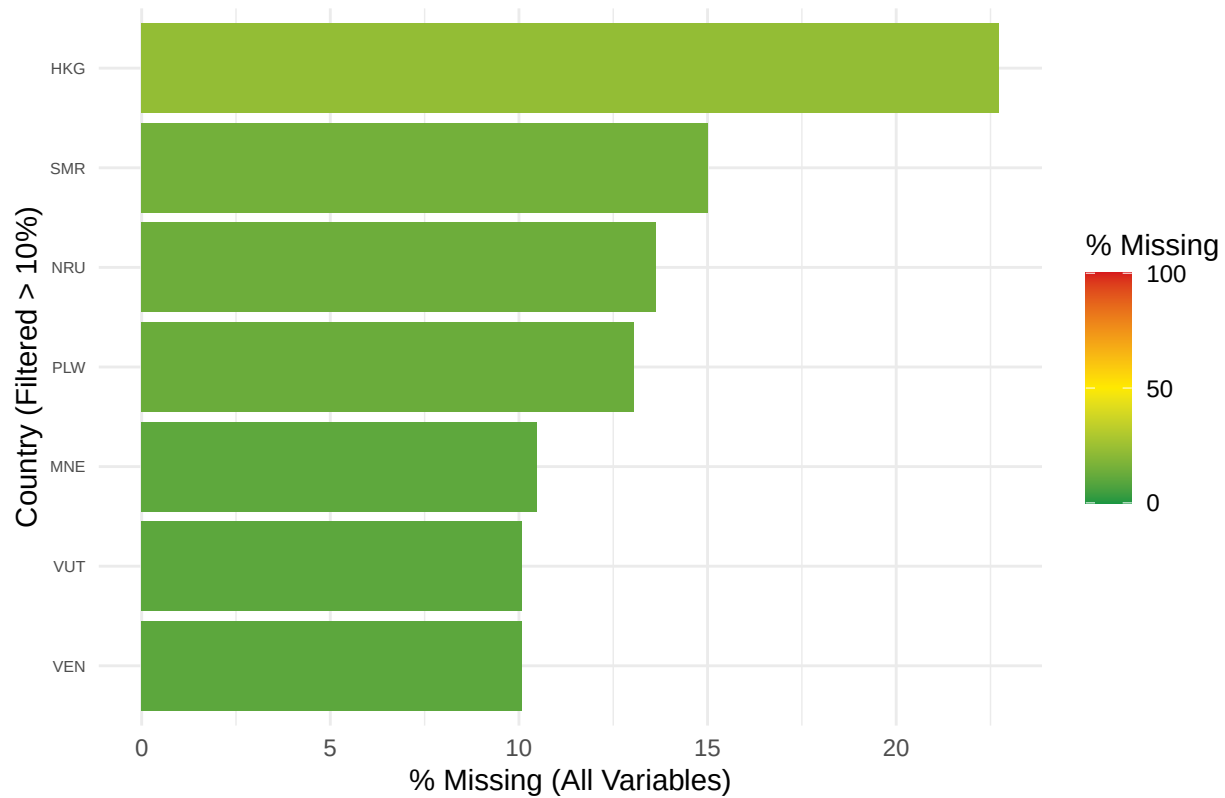
PreProcessingData <-impute_time_trend(PreProcessingData, WaterStress, 0,Inf)
PreProcessingData <-impute_time_trend(PreProcessingData, VoiceAccountability, -2.5, 2.5)
PreProcessingData <-impute_time_trend(PreProcessingData, UHCServiceCoverage, 0, 100)
PreProcessingData <-impute_time_trend(PreProcessingData, RuralPopulGrowth, -Inf, Inf)
PreProcessingData <-impute_time_trend(PreProcessingData, RqdEduYears, 0, 15)
PreProcessingData <-impute_time_trend(PreProcessingData, PopInSlums, 0, 100)
PreProcessingData <-impute_time_trend(PreProcessingData, PopDensity, 0, Inf)
PreProcessingData <-impute_time_trend(PreProcessingData, PoliticalStability, -2.5,2.5)
PreProcessingData <-impute_time_trend(PreProcessingData, HealthSpend, 0, Inf)
PreProcessingData <-impute_time_trend(PreProcessingData, GovtEduSpendPctGDP, 0, 100)
PreProcessingData <-impute_time_trend(PreProcessingData, GovernanceScore, -2.5, 2.5)
PreProcessingData <-impute_time_trend(PreProcessingData, GDPPerCapGrowth, -Inf, Inf)
PreProcessingData <-impute_time_trend(PreProcessingData, GDPGrowth, -Inf, Inf)
PreProcessingData <-impute_time_trend(PreProcessingData, GDP, -Inf, Inf)
PreProcessingData <-impute_time_trend(PreProcessingData, FoodIndex,-Inf, Inf)
PreProcessingData <-impute_time_trend(PreProcessingData, ElectricAccess, 0,100)

FourPlots(PreProcessingData,10)

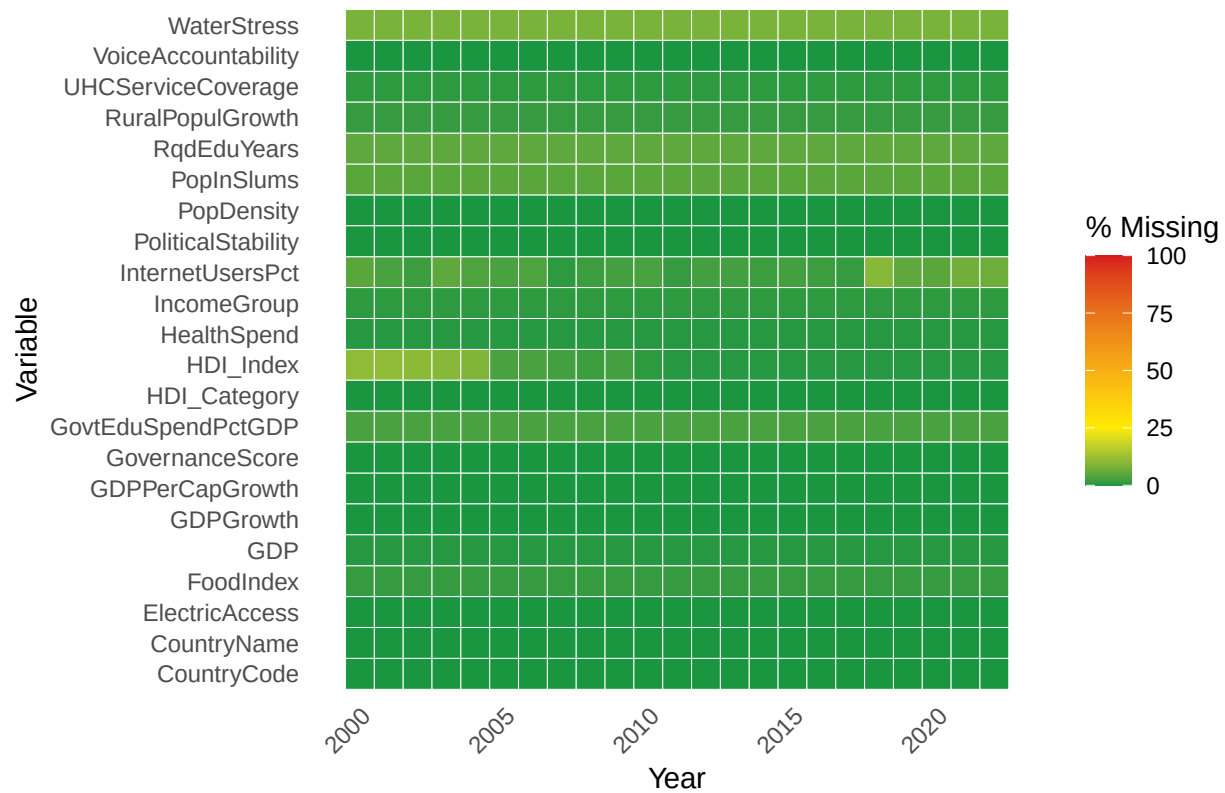
```



Data Sparseness by Country (High-Missingness Only)



Missing Data Heatmap



```

#
# DATA SET SPLITTING
#
# We now split data into training and test data. This is done as follows:
#
# 1) Keep entire countries' data together - since this is time series data, the
# observations are highly "autocorrelated" (related to nearby prior or future
# periods) and so splitting them up would create leakage.
#
# 2) Put 70% of the countries in the training data and the remaining 30% in
# the test data
#
# 3) Maintain the same stratification by IncomeGroup in both training and test data,
# since we showed in EDA that IncomeGroup is a very meaningful clustering
# differentiator for internet access and HDI - this prevents model bias
#
train_countries <- PreProcessingData %>%
  distinct(CountryCode, IncomeGroup) %>%
  group_by(IncomeGroup) %>%
  reframe(CountryCode = {
    set.seed(123) # reset for each group
    sample_frac(pick(everything()), 0.7)$CountryCode
  }) %>%
  ungroup() %>%
  pull(CountryCode)

test_countries <- setdiff(
  unique(PreProcessingData$CountryCode),
  train_countries
)

train_data <- PreProcessingData %>% filter(CountryCode %in% train_countries)
test_data <- PreProcessingData %>% filter(CountryCode %in% test_countries)
dim(train_data)

## [1] 3059 30

dim(test_data)

## [1] 1334 30

# Prepare training data distribution
train_dist <- train_data %>%
  distinct(CountryCode, IncomeGroup) %>%
  count(IncomeGroup, name = "Training_N") %>%
  ungroup() %>%
  mutate(Training_Pct = round(100 * Training_N / sum(Training_N), 1))

# Prepare test data distribution
test_dist <- test_data %>%
  distinct(CountryCode, IncomeGroup) %>%
  count(IncomeGroup, name = "Test_N") %>%
  ungroup() %>%
  mutate(Test_Pct = round(100 * Test_N / sum(Test_N), 1))

# Combine into one tibble

```

```
combined_dist <- full_join(train_dist, test_dist, by = "IncomeGroup") %>%
  arrange(IncomeGroup)
```

```
print(combined_dist)
```

```
## # A tibble: 5 x 5
##   IncomeGroup      Training_N Training_Pct Test_N Test_Pct
##   <ord>          <int>         <dbl> <int>    <dbl>
## 1 Low income           16           12      7     12.1
## 2 Lower middle income   35          26.3     15     25.9
## 3 Upper middle income   37          27.8     16     27.6
## 4 High income          44          33.1     19     32.8
## 5 <NA>                 1           0.8      1      1.7
```

```
#####
```

```
#
```

```
# REMAINING PREPROCESSING
```

```
#
```

```
# Now we do all the transformations and imputations that have to be done on training
# and test data separately to avoid leakage
```

```
# Fix the skewness
```

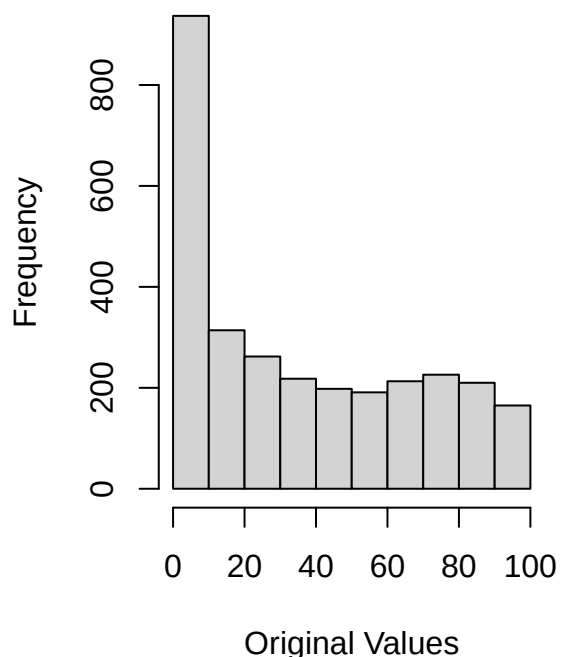
```
YJ_result <- Yeo_Johnson(train_data, test_data)
```

```
##
```

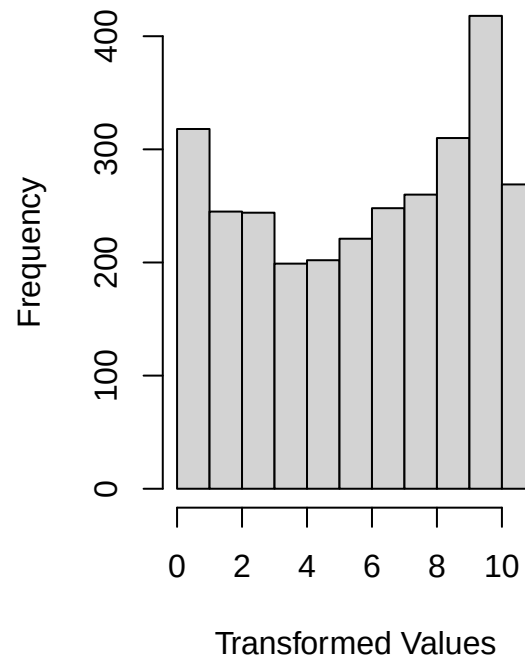
```
## Estimated lambdas for transformed variables:
```

```
##   InternetUsersPct  Lag1_InternetUsersPct  Lag2_InternetUsersPct
##           0.3248512           0.2690557           0.2148587
##   YearlyChgInternet Lag1_YearlyChgInternet Lag2_YearlyChgInternet
##           0.7629159           0.7510893           0.7470847
```

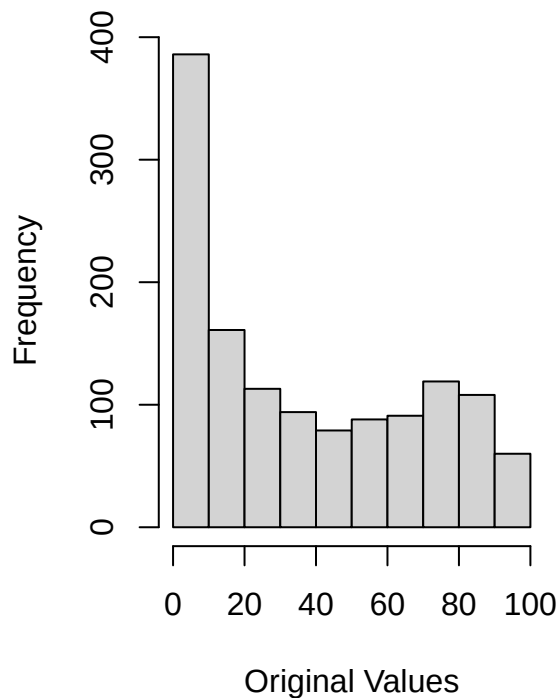
Train: Original Internet Users



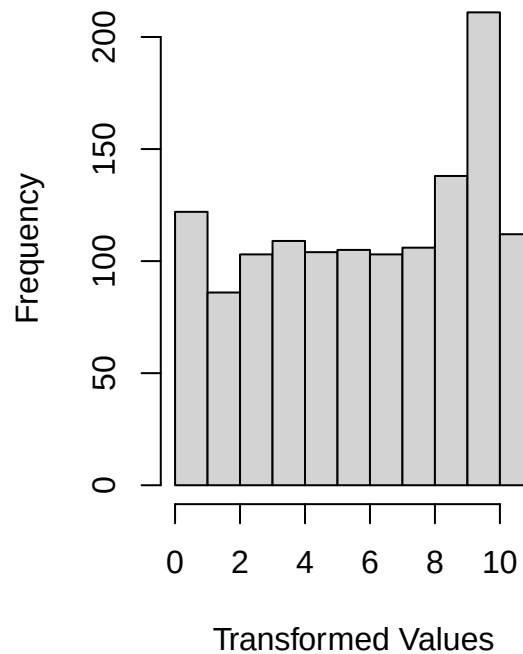
Train: Transformed Internet User



Test: Original Internet Users



Test: Transformed Internet Users



```
##
## First 5 rows of Transformed Training Data:
## # A tibble: 6 x 30
##   CountryName CountryCode year HDI_Index HDI_Category FoodIndex ElectricAccess
##   <chr>         <chr>    <int>    <dbl> <fct>         <dbl>         <dbl>
## 1 Antigua and~ ATG      2022    0.848 Very high h~    90.0         100
## 2 Antigua and~ ATG      2000    NaN    Low human d~    141.         97.7
## 3 Antigua and~ ATG      2001    NaN    Low human d~    144.         97.7
## 4 Antigua and~ ATG      2002    NaN    Low human d~    144.         100
## 5 Antigua and~ ATG      2003    NaN    Low human d~    150.         97.9
## 6 Antigua and~ ATG      2004    NaN    Low human d~    153         97.9
## # i 23 more variables: PopDensity <dbl>, PopInSlums <dbl>, WaterStress <dbl>,
## #   InternetUsersPct <dbl>, GDPPerCapGrowth <dbl>, GDPGrowth <dbl>, GDP <dbl>,
## #   PoliticalStability <dbl>, RqdEduYears <dbl>, GovtEduSpendPctGDP <dbl>,
## #   UHCServiceCoverage <dbl>, RuralPopulGrowth <dbl>,
## #   VoiceAccountability <dbl>, IncomeGroup <ord>, YearlyChgInternet <dbl>,
## #   YearlyChgHDI <dbl>, Lag1_InternetUsersPct <dbl>,
## #   Lag2_InternetUsersPct <dbl>, Lag1_YearlyChgInternet <dbl>, ...
```

```
train_data <- YJ_result$train
test_data  <- YJ_result$test
#
#####
#
# PCA - needs to be done on training data, with the same loadings applied
# to both training and test data
# PCA Recipe
numeric_cols_for_pca <- train_data %>%
  select(where(is.numeric), -year,
         -HDI_Index,
```

```

    -Lag1_InternetUsersPct,
    -Lag2_InternetUsersPct,
    -Cumulative3yrChg_InternetUsersPct,
    -Lag2_YearlyChgInternet,
    -Lag1_YearlyChgInternet,
    -YearlyChgInternet,
    -YearlyChgHDI) %>%
  names()

pca_recipe <- train_data %>%
  recipe() %>%
  step_impute_mean(all_of(numeric_cols_for_pca)) %>%
  step_normalize(all_of(numeric_cols_for_pca)) %>%
  step_pca(all_of(numeric_cols_for_pca), num_comp = 5, id = "pca")

pca_results <- prep(pca_recipe)

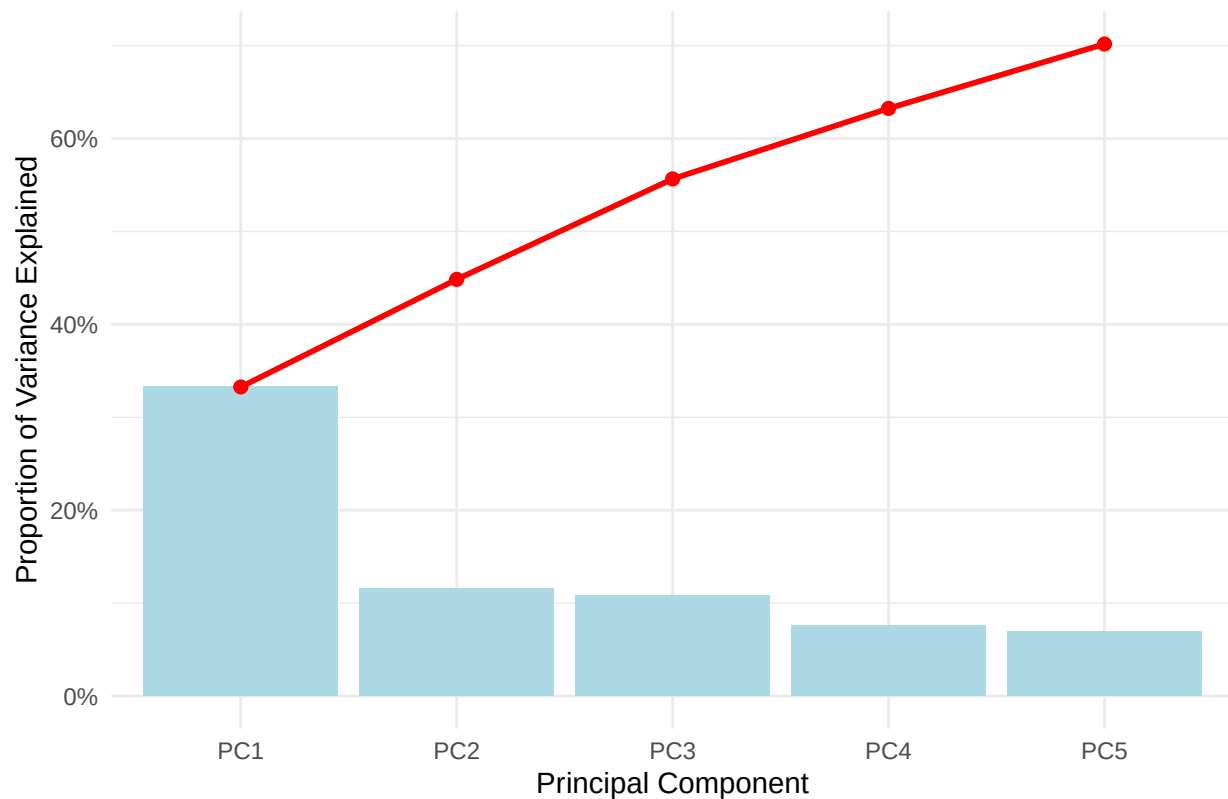
#PCA Data Baking
PCADData_train <- bake(pca_results, new_data = train_data)
PCADData_test <- bake(pca_results, new_data = test_data)

# Visualizing the PCA Data
#
# Scree Plot
# Get variance explained from the tidy output
pca_var <- tidy(pca_results, id = "pca", type = "variance") %>%
  filter(component <= 5) %>%
  pivot_wider(names_from = terms, values_from = value) %>%
  mutate(PC = paste0("PC", component),
         pct_var = `percent variance` / 100,
         cum_pct_var = `cumulative percent variance` / 100)

ggplot(pca_var, aes(x = PC)) +
  geom_col(aes(y = `pct_var`, fill = "lightblue")) +
  geom_line(aes(y = `cum_pct_var`, group = 1), color = "red", linewidth = 1) +
  geom_point(aes(y = `cum_pct_var`, color = "red", size = 2)) +
  scale_y_continuous(labels = percent_format()) +
  labs(
    title = "Scree Plot with Cumulative Variance",
    x = "Principal Component",
    y = "Proportion of Variance Explained"
  ) +
  theme_minimal()

```

Scree Plot with Cumulative Variance



```
# PCA Loadings Results
pca_loadings_long <- tidy(pca_results, id = "pca", type = "coef")
pca_loadings_wide <- pca_loadings_long %>%
  pivot_wider(
    names_from = component,
    values_from = value
  )

print("\nLoadings for each principal component:")
```

```
## [1] "\nLoadings for each principal component:"
```

```
print(pca_loadings_long)
```

```
## # A tibble: 289 x 4
##   terms          value component id
##   <chr>         <dbl> <chr>   <chr>
## 1 FoodIndex     0.105  PC1     pca
## 2 ElectricAccess 0.327  PC1     pca
## 3 PopDensity    0.0724 PC1     pca
## 4 PopInSlums    -0.337 PC1     pca
## 5 WaterStress   0.0466 PC1     pca
## 6 InternetUsersPct 0.343  PC1     pca
## 7 GDPPerCapGrowth -0.0438 PC1     pca
## 8 GDPGrowth     -0.0887 PC1     pca
## 9 GDP           0.101  PC1     pca
## 10 PoliticalStability 0.303  PC1     pca
## # i 279 more rows
```



```

# PCA Biplot for HDI Category
PCADData_train$HDI_Category <- factor(
  PCADData_train$HDI_Category,
  levels = c("Very high human development", "High human development",
             "Medium human development", "Low human development")
)

pca_biplot <- ggplot(PCADData_train, aes(x = PC1, y = PC2)) +

  geom_hline(yintercept = 0, linetype = "dashed", color = "gray") +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray") +

  geom_point(alpha = 0.5, aes(color = HDI_Category)) +

  geom_segment(
    data = pca_loadings_wide,
    aes(x = 0, y = 0, xend = PC1 * 24, yend = PC2 * 24),
    arrow = arrow(length = unit(0.3, "cm")),
    color = "gray",
    linewidth = 0.5
  ) +

  geom_text(
    data = pca_loadings_wide,
    aes(x = PC1 * 28, y = PC2 * 32, label = terms),
    color = "black",
    size = 2,
    hjust = 0.5, vjust = 3
  ) +

  labs(
    title = "PCA Biplot of PC1 and PC2",
    x = "Principal Component 1",
    y = "Principal Component 2"
  ) +

  coord_fixed(ratio = 1) +
  scale_x_continuous(limits = c(-12, 12)) +
  scale_y_continuous(limits = c(-12, 12)) +

  theme_minimal()

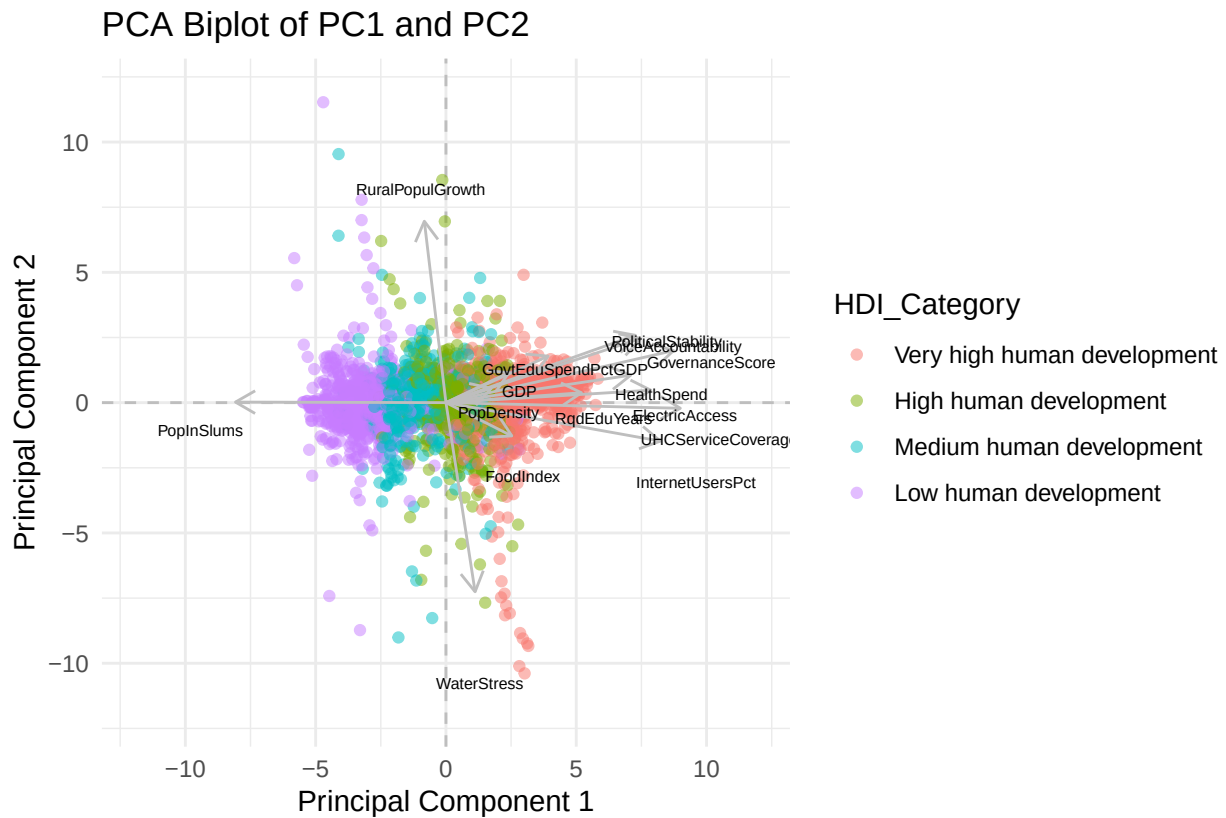
print(pca_biplot)

## Warning: Removed 3 rows containing missing values or values outside the scale range
## (`geom_point()`).

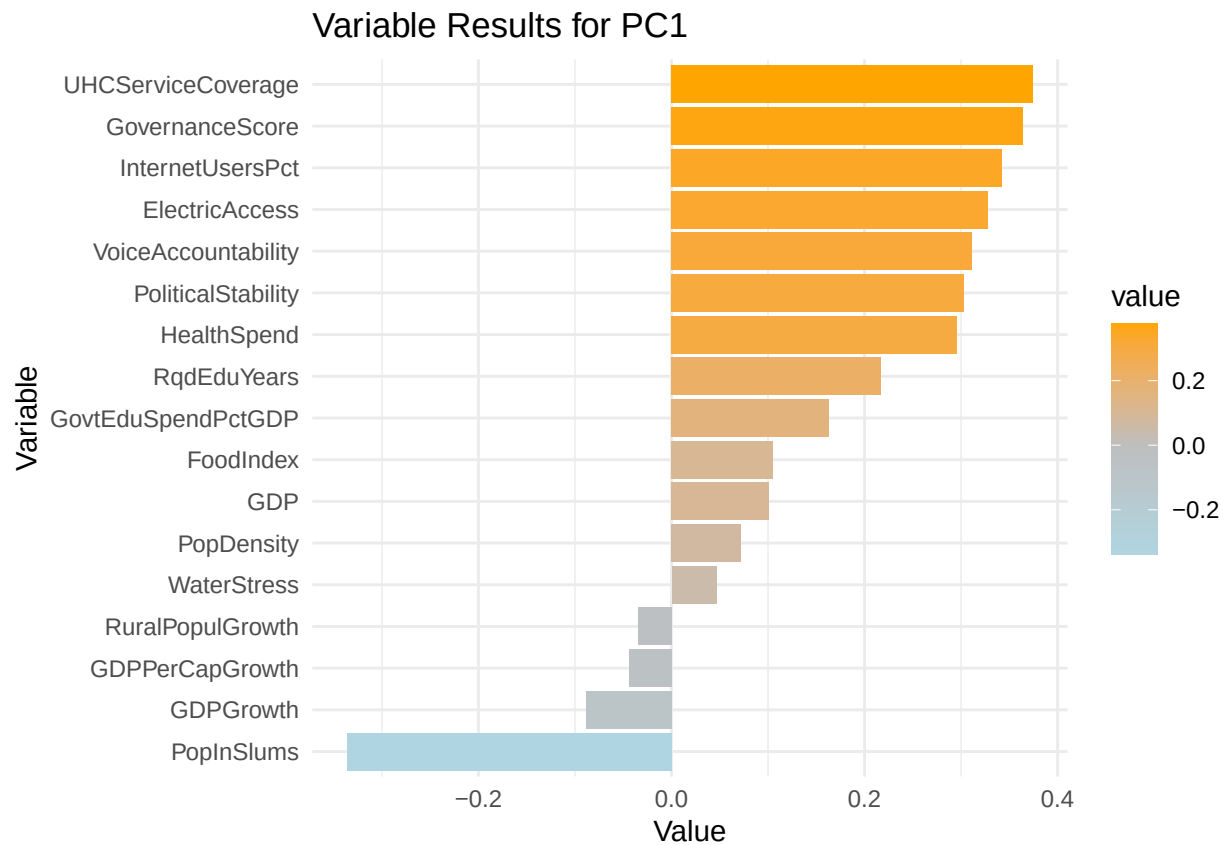
## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_segment()`).

## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_text()`).

```



```
#PCA Bar Chart
pca_bar_plot <- pca_loadings_long %>%
  filter(component == "PC1") %>%
  ggplot(aes(x = reorder(terms, value), y = value, fill = value)) +
  geom_col() +
  scale_fill_gradient2(low = "lightblue", high = "orange", mid = "gray",
    midpoint = 0) +
  coord_flip() +
  labs(
    title = "Variable Results for PC1",
    x = "Variable",
    y = "Value"
  ) +
  theme_minimal()
print(pca_bar_plot)
```



```
# SAVE THE PREPPED DATA FILES FOR MODELS TO INGEST
#
saveRDS(train_data, file = here::here("Data", "train_data.rds"))
saveRDS(test_data, file = here::here("Data", "test_data.rds"))
saveRDS(PCADData_test, file = here::here("Data", "PCADData_test.rds"))
saveRDS(PCADData_train, file = here::here("Data", "PCADData_train.rds"))
```